Politécnico do Porto
Instituto Superior de Engenharia do Porto

Sistema para Agendamento de Utilização de Laboratório Remoto

Diogo André Dias de Sousa

LEEC
Licenciatura em Engenharia Eletrotécnica e de Computadores



Departamento de Engenharia Eletrotécnica
Instituto Superior de Engenharia do Porto

Setembro, 2026

Este relatório satisfaz, parcialmente, os requisitos que constam da Ficha de Unidade Curricular de Projeto/Estágio, do 3º ano, da Licenciatura em Engenharia Eletrotécnica e de Computadores.

Candidato: Diogo André Dias de Sousa, Nº 1220982, 1220982@isep.ipp.pt

Orientação Científica: Jorge Manuel Estrela da Silva, jes@isep.ipp.pt

Departamento de Engenharia Eletrotécnica
Instituto Superior de Engenharia do Porto
Rua Dr. António Bernardino de Almeida, 431, 4200–072 Porto

Setembro, 2026

Aos que me acompanharam ao longo deste percurso.

Agradecimentos

A realização deste trabalho representa o culminar de quatro anos de licenciatura e uma meta muito importante a nível pessoal, pelo que não podia deixar de expressar o meu agradecimento a todas as pessoas que, de alguma forma, marcaram este percurso. Deste modo, quero agradecer:

Ao Instituto Superior de Engenharia do Porto, por se ter tornado uma segunda casa ao longo destes quatro anos, e a todos os professores que, de boa vontade, se disponibilizaram nos momentos em que precisei de ajuda.

Ao meu orientador, o Professor Jorge Estrela, por se ter mostrado sempre disponível, por responder às minhas dúvidas com uma prontidão que nunca deixou o trabalho parado e por ter acompanhado o projeto do princípio ao fim com grande interesse. Se foi exigente, deu-me também a autonomia para chegar às soluções por mim próprio, e é em boa parte a ele que se deve o resultado final.

A toda a minha família, em especial aos meus pais e avós, por se mostrarem sempre preocupados com o meu percurso e por acreditarem piamente nas minhas capacidades.

À minha namorada, que melhor do que ninguém acompanhou de perto todo este processo. Esteve sempre preocupada e pronta a ajudar, acreditou em mim e transmitiu-me positividade nos dias em que ela escasseava.

Aos pais da minha namorada, pela forma como me acolheram, pelo interesse constante com que acompanharam esta etapa e pelo apoio que nunca me deixaram faltar.

A todos os meus amigos que me acompanharam durante a licenciatura, sempre prontos a ajudar e a partilhar o conhecimento que iam adquirindo ao longo do curso, e também aos amigos de fora do contexto académico, que também nunca deixaram de me apoiar.

Resumo

Os laboratórios remotos permitem que estudantes de engenharia utilizem equipamento físico real através da Internet, sem necessidade de estarem presentes nas instalações. No Departamento de Engenharia Eletrotécnica (DEE) do Instituto Superior de Engenharia do Porto (ISEP) existe um laboratório deste tipo, baseado num *Raspberry Pi*, cujo acesso necessitava de um sistema capaz de gerir as reservas e controlar quem pode utilizar o equipamento e em que momento.

Este relatório descreve o desenvolvimento de um sistema de agendamento para um laboratório remoto, constituído por três componentes: uma *Application Programming Interface* (API) *backend*, desenvolvida com *Node.js* e *Express* e suportada por uma base de dados *Structured Query Language* (SQL); uma aplicação *frontend* desenvolvida com *React*, utilizada para as reservas e para a administração do sistema; e alterações a uma aplicação web previamente existente, executada num *Raspberry Pi*, responsável pelo controlo do *hardware* do laboratório, incluindo câmara e comunicação série assíncrona.

A autenticação dos utilizadores é realizada através do mecanismo de *Single Sign-On* (SSO) do DEE. O sistema valida o *JSON Web Token* (JWT) recebido através das chaves públicas disponibilizadas no *JSON Web Key Set* (JWKS) e, após essa validação, emite um *token* de sessão local para autorizar os pedidos à API. O acesso ao laboratório é gerido através de um sistema de créditos semanais, reservas com duração de 10, 20 ou 30 minutos e mecanismos de deteção de conflitos. Quando um horário fica disponível, os utilizadores em espera podem receber temporariamente a vaga através da fila de espera. O sistema inclui ainda tarefas em segundo plano para finalizar sessões, registar faltas e desligar o equipamento no final das reservas ou quando estas são terminadas antecipadamente.

Foi implementada também uma fila de espera em tempo real, complementando a fila por email. Este mecanismo atribui as vagas libertadas de forma sequencial e monitoriza a presença dos utilizadores em espera através de sinais de vida periódicos (*heartbeat*). Foram ainda adicionados mecanismos para terminar automaticamente sessões abandonadas e suporte a vários postos de trabalho.

O resultado é uma solução de baixo custo que reutiliza a infraestrutura de autenticação existente no DEE e que se encontra atualmente em funcionamento.

Palavras-Chave: Laboratório remoto, Sistema de reservas, Aplicação web.

Abstract

Remote laboratories allow engineering students to use real physical equipment over the Internet, without the need to be on-site. The DEE at ISEP features a laboratory of this type, based on a *Raspberry Pi*, whose access required a system capable of managing reservations and controlling who can use the equipment and at what time.

This report describes the development of a scheduling system for a remote laboratory, consisting of three components: a *backend* API, developed with *Node.js* and *Express* and backed by an SQL database; a *frontend* application developed with *React*, used for reservations and system administration; and modifications to a pre-existing web application, running on a *Raspberry Pi*, responsible for controlling the laboratory's hardware, including the camera and asynchronous serial communication.

User authentication is performed through the DEE SSO mechanism. The system validates the received JWT using the public keys provided in the JWKS and, following this validation, issues a local session token to authorize requests to the API. Laboratory access is managed through a weekly credit system, reservations with durations of 10, 20, or 30 minutes, and conflict detection mechanisms. When a time slot becomes available, waitlisted users can be temporarily offered the vacancy. The system also includes background tasks to finalize sessions, register no-shows, and shut down the equipment at the end of reservations or when they are terminated early.

A real-time waitlist was also implemented, complementing the email queue. This mechanism offers released slots in a staggered manner, following the order of arrival in the queue, and monitors the presence of waiting users through periodic activity signals (*heartbeats*). Furthermore, mechanisms were added to automatically terminate abandoned sessions and support multiple workstations.

The result is a low-cost solution that reuses the existing authentication infrastructure at the DEE and is currently fully operational.

Keywords: Remote laboratory, Scheduling system, Web application.

Índice

Lista de Figuras **ix**

Lista de Tabelas **xi**

Listagens **xiv**

Lista de Algoritmos **xv**

Glossário **xvii**

Lista de Acrónimos **xix**

1 Introdução **1**
- 1.1 Contextualização 1
- 1.2 Descrição do Projeto e Objetivos 2
  - 1.2.1 Objetivos 2
- 1.3 Calendarização 3
- 1.4 Organização do Relatório 3

2 Estado da Arte **5**
- 2.1 Sistemas de Agendamento e Gestão de Recursos Partilhados 5
- 2.2 Controlo de Concorrência, Filas de Espera e Monitorização de Presença 6
- 2.3 Autenticação Federada e Validação de Sessões: SSO, JWT e JWKS . 7
- 2.4 Arquiteturas Web: *Single Page Application* e API *Representational State Transfer* (REST) 7

3 Arquitetura do Sistema **9**
- 3.1 Visão Geral 9
- 3.2 Modelo de Dados 10
- 3.3 Fluxo de Autenticação 12
- 3.4 Regras de Reserva e Créditos 13
- 3.5 Tarefas em Segundo Plano 15
- 3.6 Considerações de Configuração e Segurança 16
- 3.7 Acesso ao Raspberry Pi: Token de Sessão e Canal de Revogação . . 16
- 3.8 Suporte a Múltiplos Postos de Trabalho 17

4 Implementação do Backend **19**
4.1 Estrutura e Tecnologias 20
4.2 Principais Rotas da API 21
4.3 Deteção de Conflitos de Horário 22
4.4 Encerramento Seguro do Equipamento 24
4.5 Envio de Notificações por Email 25
4.6 Estado de Falta e Deteção de Não-Comparência 26
4.7 Controlo de Hardware pelo Administrador 28
4.8 Emissão do Token de Sessão e Canal de Revogação 29
4.9 Persistência do Reset Semanal de Créditos 31
4.10 Fila de Espera em Tempo Real 32
4.11 Deteção de Abandono de Sessão 35
4.12 Suporte a Múltiplos Postos de Trabalho 36

5 Implementação do Frontend **39**
5.1 Estrutura da Aplicação 39
5.2 Calendário de Reservas 40
5.2.1 Fila de Espera em Tempo Real 42
5.3 Painel de Controlo do Laboratório 44
5.4 Painel de Administração 46
5.4.1 Acompanhamento da Sessão em Curso e Controlo de Hardware 48
5.4.2 Responsividade e Modal de Detalhe de Reserva 49
5.5 Suporte Multilingue 50

6 Integração com o Raspberry Pi **51**
6.1 Validação de Reservas no Laboratório 51
6.2 Acesso Direto para Utilizadores Autorizados 53
6.2.1 Lista de Utilizadores Autorizados 53
6.2.2 Autenticação e Validação do Token do DEE 54
6.2.3 Independência do Sistema de Reservas 55
6.3 Vídeo em Direto e Leitura da Porta Série 55
6.4 Controlo de Hardware pelo Administrador 56
6.5 Otimização do Consumo de CPU da Câmara 57
6.6 Sincronização do Estado do Hardware entre Sessões 58
6.7 Deteção de Abandono a partir da Própria Página do Laboratório . . 59

7 Testes e Resultados **61**
7.1 Estratégia de Testes 61
7.2 Cenários de Teste de Ponta a Ponta 61
7.3 Problemas Identificados e Corrigidos 62
7.4 Validação em Ambiente Real 63

8 Conclusões **65**
8.1 Trabalho Futuro 65

Referências **67**

Anexo A Excertos de Código Complementares **69**
A.1 Rota de Entrada da Aplicação do Raspberry Pi 69
A.2 Variáveis de Ambiente do Backend 71

Lista de Figuras

1.1 Cronograma das principais fases do projeto. 3

3.1 Arquitetura geral do sistema. O *browser* comunica sempre através do *proxy* reverso do Departamento de Engenharia Eletrotécnica (DEE); o *backend* valida a autenticação junto do DEE e comunica diretamente com o *Raspberry Pi* para validar acessos e desligar o equipamento; o *Raspberry Pi* valida também, de forma independente, os utilizadores com acesso direto autorizado (Secção 6.2). 10

3.2 Diagrama entidade-associação da base de dados, com as chaves estrangeiras que ligam `reservas`, `fila_espera` e `vaga_pendente` à tabela `utilizadores`, e `reservas` à tabela `postos`. 12

3.3 Sequência de acesso ao *Raspberry Pi*. O *token* de sessão é emitido pelo *backend* na confirmação de entrada e validado localmente pelo equipamento; o canal *Server-Sent Events* (SSE) aberto a seguir permite revogar a sessão antes do prazo de validade do *token*. 17

4.1 Email de notificação enviado ao primeiro aluno da fila de espera quando uma vaga é libertada, com a hora-limite para a marcação e a ligação direta para o dia atribuído. 26

4.2 Revelação progressiva de uma vaga libertada. O aluno na posição n da fila recebe a oferta ao fim de $T_1 \times (n-1)$ segundos e mantém-na até alguém aceitar, pelo que as ofertas se acumulam em vez de se substituírem: quem está mais à frente na fila tem uma vantagem de T_1 segundos sobre o seguinte, e não a exclusividade da escolha. . . . 33

5.1 Calendário de reservas do aluno: seleção de duração, dia (colorido consoante a disponibilidade) e período do dia, com o resumo de créditos e o seletor de língua no cabeçalho. 41

5.2 Aviso apresentado ao aluno a quem a fila de espera atribuiu uma vaga (Secção 4.5): o dia fica reservado só para ele durante uma hora, com contagem decrescente do prazo de marcação e o dia assinalado no calendário. 42

5.3 Aviso apresentado enquanto o aluno aguarda na fila em tempo real, com a posição atual, atualizada sempre que outro aluno entra, desiste ou avança, e a opção de sair da fila. 43
5.4 Oferta de vaga recebida pelo canal SSE, que substitui o aviso de espera. Indica o horário e a duração concretos da vaga e permite aceitá-la de imediato ou passar ao aluno seguinte. 43
5.5 Painel de controlo do laboratório nos estados `"contagem"` (em cima, com o botão de entrada bloqueado até à hora reservada) e `"ativo"` (em baixo, sessão já em curso). 44
5.6 Painel de administração — separador "Reservas", com a navegação lateral e a listagem de todas as reservas do sistema, filtrável por data e por estado. 47
5.7 Separador "Utilizadores": listagem das contas registadas, com o *username* institucional, o papel atribuído, os créditos disponíveis e a ação de ajuste de créditos. 47
5.8 Separador "Estatísticas": totais de reservas ativas, concluídas, canceladas, faltas e utilizadores, acompanhados do tempo de utilização por aluno e da distribuição das sessões por hora do dia. 47
5.9 Separador "Configurações": parâmetros da fila em tempo real (T_1 e janela de verificação), do limite de sessão abandonada, e gestão dos postos de trabalho. 48
5.10 Separador "Posto ao Vivo": vídeo em direto, estado dos interruptores e saída do DAC (em cima), e consola série com histórico e campo de envio (em baixo). 49

Lista de Tabelas

3.1 Tabelas principais do modelo de dados. 11
3.2 Principais campos da tabela `reservas`. 11

4.1 Principais operações da *Application Programming Interface* (API) consumidas pela aplicação *frontend*. 21
4.2 Operações da API consumidas pela aplicação do laboratório, no *Raspberry Pi*. 22

7.1 Cenários de teste de ponta a ponta validados manualmente. 62
7.2 Principais problemas identificados durante o desenvolvimento e validação do sistema. 63
7.3 Ocupação do processador do Raspberry Pi, medida com `top`, antes e depois da otimização da câmara. 64

Listagens

4.1 Módulo de configuração do *backend*, que reúne os parâmetros de implantação que não constituem segredos. 20
4.2 Instrução SQL de deteção de conflitos de horário, com o parâmetro `cruzouMeiaNoite` ligado ao Cenário 3. 23
4.3 Função auxiliar responsável por avisar o Raspberry Pi para desligar o equipamento antes de uma reserva ser fechada. 24
4.4 Rota de cancelamento antecipado, agora a avisar sempre o Raspberry Pi antes de fechar a reserva na base de dados (excerto simplificado). 25
4.5 Tarefa de deteção de faltas, com o tratamento das reservas que atravessam a meia-noite (excerto simplificado). 27
4.6 Função auxiliar que reencaminha um comando de hardware para o Raspberry Pi, autenticado pelo segredo partilhado entre backend e Pi. 28
4.7 Cálculo do prazo de validade do token de sessão, com o mesmo tratamento de reservas que atravessam a meia-noite usado no resto do sistema (excerto simplificado). 30
4.8 Endpoint de eventos de sessão: o Raspberry Pi liga-se aqui, autenticado pelo próprio token, e a ligação fica registada para receber uma eventual revogação. 30
4.9 Verificação periódica do reset semanal, comparando a segunda-feira mais recente já passada com a data do último reset guardada na base de dados (excerto simplificado). 31
4.10 Critério que decide se um aluno deve ser encaminhado para a fila de espera: janela de verificação para hoje, dia completo para dias futuros (excerto simplificado). 32
4.11 Tarefa periódica de deteção de abandono: uma sessão sem heartbeat há mais tempo do que o limite configurado é terminada automaticamente (excerto simplificado). 35
4.12 Escolha automática de um posto livre, reutilizando a instrução SQL de deteção de conflitos já usada na criação de reservas (excerto simplificado). 36
4.13 Um bloco só é considerado cheio quando todos os postos ativos têm, nesse bloco, uma reserva (excerto simplificado). 37

5.1 Função utilitária, reutilizada em dois pontos da interface, que abre a hiperligação de composição de evento no Outlook Web apenas por ação explícita do utilizador. 41
5.2 Cálculo do estado do painel de controlo (contagem decrescente ou sessão ativa), com tratamento de reservas que atravessam a meia-noite. 44
5.3 Ação do botão de pânico: confirmação, pedido ao backend e atualização imediata do estado local. 45
6.1 Validação local do token de sessão, incluindo a verificação de revogação (Secção 3.7). 52
6.2 Escuta do canal de revogação, com reconexão automática enquanto o token ainda for válido (Secção 4.8, excerto simplificado). 52
6.3 Ficheiro de configuração com a lista de utilizadores autorizados a aceder diretamente ao laboratório. 53
6.4 Validação do JWT do DEE na aplicação do Raspberry Pi, obtendo diretamente a chave pública a partir do JWKS. 54
6.5 Verificação de acesso direto, baseada exclusivamente na sessão do browser e na lista de utilizadores autorizados. 54
6.6 Autenticação das rotas de vídeo e de porta série, exigindo um token de sessão válido ou acesso direto. 55
6.7 Verificação do segredo partilhado nos endpoints de controlo de hardware do administrador. 56
6.8 Gestão do número de espectadores ativos, usada para ligar e desligar a captura de vídeo da câmara (excerto simplificado). 57
6.9 Consulta periódica do estado do laboratório na própria página do aluno, com cache-busting para evitar respostas desatualizadas (excerto simplificado). 58
6.10 Envio periódico de heartbeat a partir da página do laboratório, usando o token de sessão como identificação (excerto simplificado). . 59
A.1 Rota de entrada da aplicação Flask do Raspberry Pi. 69
A.2 Verificação de acesso na rota principal da aplicação do laboratório (excerto simplificado). 70
A.3 Estrutura do ficheiro .env do backend (sem valores reais). 71

Lista de Algoritmos

4.1 Deteção de conflitos de horário no momento da reserva 23
4.2 Agendamento em cascata das ofertas da fila em tempo real 34

Glossário

endpoint

rota específica de uma API, identificada por um caminho *Uniform Resource Locator* (URL) e um método *HyperText Transfer Protocol* (HTTP) (por exemplo, `POST /api/reservar`), à qual o cliente envia pedidos.

middleware

camada de software que se interpõe entre componentes de um sistema, disponibilizando-lhes serviços comuns. Nas *frameworks* web, como o *Express*, designa em particular cada uma das funções que compõem a cadeia de processamento de um pedido HTTP, podendo intercetá-lo antes de este chegar à rota de destino — por exemplo, para validar um *token* de autenticação.

polling

técnica de verificação repetida e periódica do estado de um recurso (por exemplo, tarefas em segundo plano executadas a cada minuto), em alternativa a notificações assíncronas.

***proxy* reverso**

servidor intermédio que reencaminha pedidos de clientes externos para um servidor de aplicação interno, podendo alterar o caminho do pedido antes de o reencaminhar; usado na infraestrutura do ISEP para publicar o sistema sob um caminho comum.

Lista de Acrónimos

API	*Application Programming Interface*
CRA	*Create React App*
CRUD	*Create, Read, Update, Delete*
DAC	*Digital-to-Analog Converter*
DEE	Departamento de Engenharia Eletrotécnica
FPS	*Frames Per Second*
GPIO	*General Purpose Input/Output*
HTML	*HyperText Markup Language*
HTTP	*HyperText Transfer Protocol*
I2C	*Inter-Integrated Circuit*
IP	*Internet Protocol*
ISEP	Instituto Superior de Engenharia do Porto
JPEG	*Joint Photographic Experts Group*
JSON	*JavaScript Object Notation*
JWKS	*JSON Web Key Set*
JWT	*JSON Web Token*
LED	*Light Emitting Diode*
LEEC	Licenciatura em Engenharia Eletrotécnica e de Computadores
MJPEG	*Motion JPEG*
REST	*Representational State Transfer*
SGBD	Sistema de Gestão de Base de Dados

SPA	*Single Page Application*
SQL	*Structured Query Language*
SSE	*Server-Sent Events*
SSO	*Single Sign-On*
UC	Unidade Curricular
URL	*Uniform Resource Locator*
XHR	*XMLHttpRequest*

Capítulo 1

Introdução

Este capítulo contextualiza o projeto do sistema de agendamento do laboratório remoto de MICMIC, apresenta os objetivos propostos, a cronologia do desenvolvimento do projeto e a organização do relatório.

1.1 Contextualização

Os laboratórios remotos constituem uma solução eficaz para disponibilizar acesso a equipamento físico dispendioso ou de disponibilidade limitada a um número alargado de estudantes, sem exigir presença física nas instalações. Porém, colocam desafios complexos ao nível do agendamento e da gestão do tempo de utilização. Uma vez que o *hardware* físico é um recurso finito e exclusivo, a inexistência de um sistema automatizado de reservas compromete a sua rentabilização e dificulta o acesso equitativo por parte dos alunos.

No DEE do Instituto Superior de Engenharia do Porto (ISEP), encontrava-se já operacional uma bancada de laboratório remoto baseada num *Raspberry Pi* [1], destinada ao controlo e monitorização de equipamento didático (simulação dos interruptores e entradas digitais, *Light Emitting Diode*s (LEDs), mostradores de 7 segmentos, câmara de vídeo e porta série). Contudo, a utilização desta infraestrutura pré-existente assentava numa gestão informal e manual. A ausência de uma camada de gestão dedicada resultava em vários problemas operacionais: falta de controlo de acessos e de autenticação institucional, impossibilidade de garantir uma distribuição justa do tempo de bancada, ocorrência de conflitos por sobreposição de

utilizadores e ausência de mecanismos automáticos para a terminação de sessões e encerramento do equipamento.

A bancada é constituída por um *Raspberry Pi* ligado à placa de desenvolvimento do microcontrolador *ATmega128* e à placa de periféricos utilizada nas aulas de MICMIC, que integra oito LEDs, quatro mostradores de 7 segmentos e seis botões de pressão. O *Raspberry Pi* substitui a intervenção física do aluno sobre essa placa: um expansor de entradas/saídas, ligado por barramento *Inter-Integrated Circuit* (I2C), aciona as entradas digitais correspondentes aos botões, enquanto um conversor digital-analógico, no mesmo barramento, impõe o valor de tensão lido pela entrada analógica do microcontrolador. A adaptação entre os níveis lógicos do *Raspberry Pi* e os da placa é assegurada por conversores de nível.

Sobre esta montagem, o aluno pode enviar o *firmware* que compilou, gravado no *ATmega128* pelo *Raspberry Pi* através da interface de programação em série, comunicar com o microcontrolador pela porta série, e observar o resultado — o estado dos LEDs e dos mostradores — através de uma câmara de vídeo apontada à bancada. Tanto a montagem como a aplicação web que a controla foram desenvolvidas anteriormente, no âmbito de outro projeto de PESTA da Licenciatura em Engenharia Eletrotécnica e de Computadores (LEEC) [2]. Essa aplicação, escrita em *Python* com recurso à *framework Flask* [3], corre no próprio *Raspberry Pi* e disponibiliza, numa página web, os comandos sobre o *hardware*, a consola série e o vídeo da câmara.

Este projeto surge para resolver estas lacunas, focando-se no desenvolvimento de uma plataforma integrada de agendamento, gestão de reservas e controlo de acessos em tempo real, articulada com o sistema de autenticação do DEE e integrada com a bancada remota existente.

1.2 Descrição do Projeto e Objetivos

O laboratório de MICMIC é constituído por três componentes principais em comunicação síncrona e assíncrona: uma API *backend* (lógica de negócio e gestão de acessos), um *frontend* em página única (*Single Page Application* (SPA), interface do utilizador) e uma aplicação alojada no *Raspberry Pi* do laboratório (gestão do *hardware*).

1.2.1 Objetivos

Para dar resposta às necessidades identificadas, definiram-se os seguintes objetivos principais:

- Integrar a autenticação com o mecanismo de *Single Sign-On* (SSO) do DEE, eliminando a gestão local de credenciais;

- Implementar a lógica de negócio na API *backend*: gestão de créditos semanais, reservas de duração fixa, prevenção de sobreposições de horários e gestão de filas de espera;
- Desenvolver uma interface web (SPA) intuitiva e responsiva para alunos (reservas e acesso em tempo real) e administradores (gestão do sistema e controlo do *hardware*);
- Garantir que o acesso físico ao *Raspberry Pi* é restrito ao período exato da reserva e assegurar o encerramento seguro do equipamento no final da sessão;
- Implementar um canal de acesso direto, autenticado e independente de reserva, para utilizadores autorizados (docentes);
- Validar o sistema em ambiente real de produção, otimizando o seu desempenho e fiabilidade.

1.3 Calendarização

O projeto foi executado ao longo de um semestre letivo, no âmbito da unidade curricular de Projeto/Estágio do 3.º ano da LEEC, dividindo-se nas fases apresentadas na Figura 1.1.

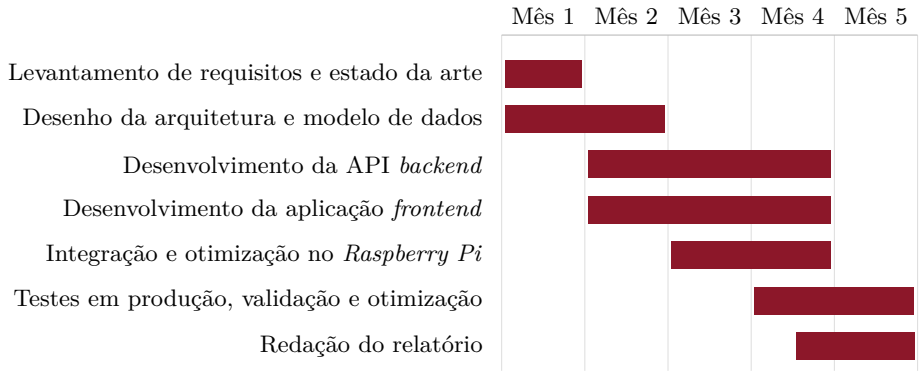


Figura 1.1: Cronograma das principais fases do projeto.

1.4 Organização do Relatório

O presente relatório encontra-se estruturado em oito capítulos:

- O **Capítulo 1** enquadra o projeto, apresentando a motivação, objetivos e organização;
- O **Capítulo 2** revê o estado da arte referente a laboratórios remotos, arquiteturas de API *Representational State Transfer* (REST) e mecanismos de autenticação (SSO/*JSON Web Token* (JWT));

- O **Capítulo 3** detalha a arquitetura global do sistema, o modelo de dados e o fluxo de autenticação;
- Os **Capítulos 4** e **5** descrevem, respetivamente, a implementação da API *backend* e da aplicação *frontend*;
- O **Capítulo 6** aborda a integração com o *Raspberry Pi*, os mecanismos de acesso direto e as otimizações de *hardware*;
- O **Capítulo 7** apresenta os testes de validação, análise de desempenho e resultados obtidos em produção;
- O **Capítulo 8** sintetiza as conclusões do trabalho e perspetiva desenvolvimentos futuros.

Capítulo 2

Estado da Arte

Este capítulo analisa os conceitos, arquiteturas e tecnologias fundamentais para o desenvolvimento do sistema de agendamento e controlo de acessos.

2.1 Sistemas de Agendamento e Gestão de Recursos Partilhados

A partilha de recursos físicos através da rede exige a definição de regras de agendamento, de forma a evitar conflitos de acesso e garantir uma distribuição justa do tempo de utilização.

Na literatura e em plataformas existentes, a gestão de acesso a recursos físicos divide-se essencialmente em três modelos:

1. **Acesso Direto (Fila de Espera Simples):** O utilizador acede ao recurso sem reserva prévia. Caso o equipamento esteja ocupado, entra numa fila sequencial. É o modelo utilizado por plataformas como o *WebLab-Deusto* em ensaios de curta duração [4].

2. **Agendamento por Bloco Fixo:** O utilizador reserva uma sessão num intervalo específico. Plataformas como o *iLab Shared Architecture* (MIT) utilizam um servidor de agendamento dedicado para gerir o calendário de reservas e prevenir sobreposições [5].

3. **Sistemas de Créditos:** Para evitar a monopolização do equipamento por parte de alguns utilizadores, associa-se a cada utilizador um saldo de tempo, descontado a cada utilização e reposto periodicamente. Nas plataformas de laboratórios remotos, este princípio surge sobretudo sob a forma de limites atribuídos a grupos de utilizadores: o *WebLab-Deusto*, por exemplo, associa a cada grupo permissões que fixam a duração máxima de cada sessão [4]. O modelo de saldo renovável é, por seu lado, corrente na partilha de recursos de computação, onde escalonadores como o *Slurm* atribuem a cada conta uma quota de tempo de processamento, consumida ao longo de um período e reposta no início do seguinte [6].

2.2 Controlo de Concorrência, Filas de Espera e Monitorização de Presença

Num recurso de capacidade unitária, o cancelamento de uma reserva ou a desistência em cima da hora deixam o equipamento livre num intervalo que dificilmente é reocupado sem intervenção do próprio sistema. O problema não é exclusivo dos laboratórios remotos — coloca-se de igual modo na marcação de salas, de equipamento científico ou de consultas —, mas agrava-se quando as sessões são curtas, porque o tempo perdido representa então uma fração significativa do total.

As plataformas existentes distinguem-se, quanto a este ponto, pelo instante em que a informação chega a quem espera, o que decorre do modelo de agendamento que adotam:

- **Notificação assíncrona:** nas plataformas assentes em reserva antecipada, como o *iLab Shared Architecture* [5], o horário libertado só é útil a quem o venha a ocupar mais tarde, pelo que o utilizador seguinte é contactado por um canal que não exige a sua presença na plataforma — tipicamente email —, sendo-lhe concedido um prazo para confirmar (*holding time*). O intervalo entre o envio e a leitura da mensagem torna esta via pouco eficaz para vagas de curta duração.

- **Notificação síncrona:** nas plataformas assentes em fila de acesso imediato, como o *WebLab-Deusto* [4], os utilizadores aguardam ligados à própria plataforma e o acesso é-lhes concedido assim que o equipamento fica livre. O atraso reduz-se à ordem dos segundos, mas o mecanismo apenas alcança quem estiver presente nesse momento.

As opções tomadas neste projeto quanto a cada um destes pontos, incluindo a razão pela qual coexistem os dois tipos de notificação, são apresentadas no Capítulo 3.

2.3 Autenticação Federada e Validação de Sessões: SSO, JWT e JWKS

A utilização de um sistema de agendamento levanta também a questão da autenticação dos utilizadores. Uma das possibilidades é recorrer a um mecanismo de SSO, frequentemente suportado por protocolos de autorização delegada como o *OAuth 2.0* [7]. Esta abordagem permite reutilizar as credenciais institucionais dos utilizadores, não sendo necessário que a aplicação mantenha uma base de dados própria com as palavras-passe.

Uma abordagem utilizada para transmitir a informação de autenticação é o JWT [8]. Este formato organiza o *token* em três partes: cabeçalho, dados e assinatura, codificadas em *Base64Url*. A assinatura permite verificar se o *token* foi emitido pelo fornecedor de identidade e se o seu conteúdo não foi alterado. Essa verificação exige ainda que a aplicação fixe explicitamente o algoritmo de assinatura esperado, evitando que um *token* assinado com um algoritmo diferente do previsto seja aceite como válido [9]. Para essa verificação podem ser utilizadas chaves públicas disponibilizadas pelo emissor através de um conjunto *JSON Web Key Set* (JWKS) [10], normalmente acessível através de um *endpoint* HTTP.

Neste projeto, a validação do JWT contra as chaves públicas do DEE permite confirmar a autenticidade do *token* sem que o sistema tenha acesso às credenciais do utilizador. Os *tokens* podem também ser utilizados para transportar informações necessárias à validação de uma sessão no equipamento remoto, evitando uma consulta à base de dados para cada pedido.

2.4 Arquiteturas Web: *Single Page Application* e API REST

Nas aplicações web tradicionais, cada interação do utilizador provoca um novo pedido ao servidor, que devolve uma página *HyperText Markup Language* (HTML) completa. O modelo SPA funciona de forma diferente: o *browser* carrega uma única vez uma aplicação *JavaScript*, tipicamente construída com uma *framework* dedicada (como o *React* [11], o *Angular* ou o *Vue*), que gere a interface localmente e comunica com o servidor apenas para trocar dados, através de uma API, tipicamente em formato *JavaScript Object Notation* (JSON).

Do lado do servidor, o estilo arquitetural REST é frequentemente utilizado neste tipo de aplicações. Cada recurso, como uma reserva ou um utilizador, é identificado por um URL, e as operações sobre esse recurso são realizadas através dos métodos HTTP: `GET` para consulta, `POST` para criação, `PUT`/`PATCH` para atualização e `DELETE` para remoção. Estes métodos correspondem às operações *Create, Read, Update, Delete* (CRUD) utilizadas na gestão dos dados.

O *Node.js* [12], através, por exemplo, da *framework Express* [13] (entre várias alternativas disponíveis), permite implementar este tipo de API, disponibilizando mecanismos para definir rotas e *middleware*s. A utilização de *JavaScript* tanto no *frontend* como no *backend* permite também manter a mesma linguagem nas duas partes da aplicação.

Capítulo 3

Arquitetura do Sistema

Este capítulo apresenta a arquitetura geral do projeto, o modelo de dados utilizado, o fluxo de autenticação e as principais regras de negócio associadas às reservas, aos créditos e às tarefas em segundo plano que mantêm o sistema a funcionar de forma autónoma.

3.1 Visão Geral

O sistema assenta numa topologia centrada num processo *Node.js* (Capítulo 4), que serve simultaneamente a aplicação *frontend* (Capítulo 5) e a API REST sob o prefixo `/api`, que comunica com a aplicação do laboratório remoto alojada no *Raspberry Pi* (descrita em mais detalhe no Capítulo 6) através de pedidos HTTP. Em produção, a aplicação opera atrás de um *proxy* reverso do DEE (no caminho `/DEE/micmic-lab`). Como tal, o *backend* encarrega-se de remover este prefixo para garantir o encaminhamento correto dos pedidos.

A Figura 3.1 resume esta arquitetura e os respetivos fluxos de comunicação. É importante notar que os fluxos representados dizem respeito apenas ao *controlo de acesso*: a utilização do laboratório propriamente dita não passa pelo *backend*. Assim que a entrada é confirmada, o *browser* do aluno é reencaminhado para o endereço do próprio *Raspberry Pi*, levando consigo o *token* de sessão que lhe dá acesso (Secção 3.7); é essa aplicação que serve, a partir daí, o vídeo, a consola série e os comandos sobre o *hardware*.

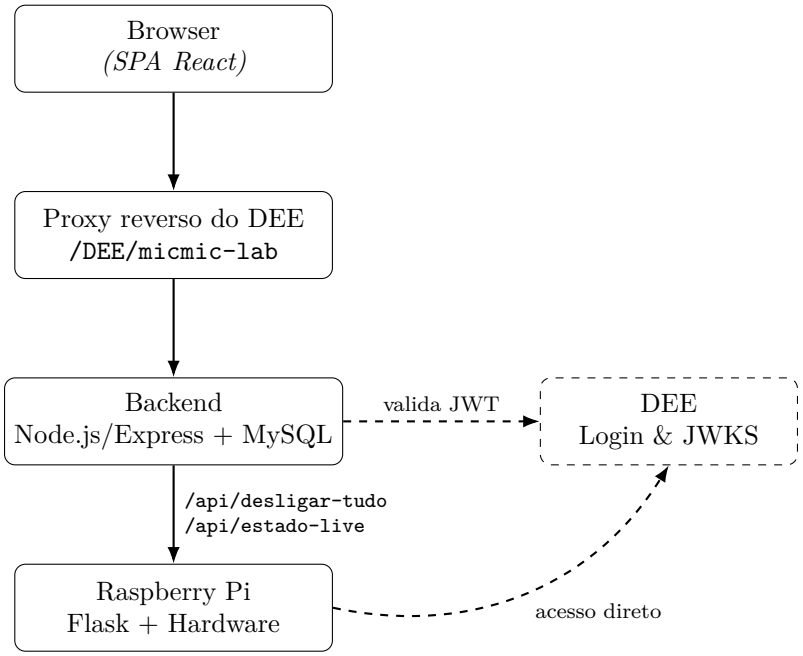


Figura 3.1: Arquitetura geral do sistema. O *browser* comunica sempre através do *proxy* reverso do DEE; o *backend* valida a autenticação junto do DEE e comunica diretamente com o *Raspberry Pi* para validar acessos e desligar o equipamento; o *Raspberry Pi* valida também, de forma independente, os utilizadores com acesso direto autorizado (Secção 6.2).

Esta arquitetura simplifica a disponibilização do sistema e minimiza os saltos de rede entre os pedidos e a sua execução. Contudo, a dependência num único processo implica que, em caso de reinício do servidor, os temporizadores das tarefas em segundo plano (Secção 3.5) tenham de ser redefinidos a partir desse instante.

3.2 Modelo de Dados

A persistência do sistema assenta numa base de dados alojada num servidor *MySQL* [14]. As consultas à base de dados são feitas através de *Structured Query Language* (SQL), suportada por este Sistema de Gestão de Base de Dados (SGBD). As tabelas são criadas pelo *backend* na sua primeira execução. A Tabela 3.1 resume as tabelas do sistema e as suas respetivas funções.

Tabela 3.1: Tabelas principais do modelo de dados.

Tabela	**Descrição**
`utilizadores`	Conta de cada aluno/administrador: papel (`aluno` ou `admin`), créditos disponíveis (em minutos) e data/hora do fim da última reserva.
`reservas`	Cada reserva efetuada: utilizador, data, hora de início e fim, e estado atual (este aspeto será detalhado na Secção 3.4).
`fila_espera`	Lista de alunos em espera por disponibilidade num determinado dia, quando este se encontra completamente reservado. Suporta os mecanismos de fila por email (dias futuros) e de fila em tempo real (dia atual), descritos na Secção 3.4.
`vaga_pendente`	Vagas pendentes: reservas temporárias, com prazo de expiração, criadas quando uma vaga se liberta para o primeiro aluno da fila de espera por email.
`postos`	Postos de trabalho existentes no laboratório — nome (a designação do posto), endereço IP e porto HTTP do respetivo *Raspberry Pi*, e se está ativo ou não (Secção 3.8).
`estado_sistema`	Estado interno e parâmetros configuráveis do sistema — a data da última reposição semanal de créditos (Secção 3.5), os parâmetros da fila em tempo real e o tempo de inatividade antes de uma sessão ser considerada abandonada (Secção 3.4).

A Tabela 3.2 detalha os campos mais relevantes da tabela `reservas`, o núcleo da lógica de negócio do sistema.

Tabela 3.2: Principais campos da tabela `reservas`.

Campo	**Descrição**
`id`	Identificador único da reserva.
`utilizador_id`	Aluno a quem pertence a reserva.
`data_reserva`	Data da sessão reservada.
`hora_inicio` / `hora_fim`	Hora de início e fim da sessão (Secção 3.4).
`posto_id`	Posto de trabalho atribuído à reserva, escolhido automaticamente entre os postos ativos (Secção 3.8).
`estado`	Fase atual do ciclo de vida da reserva: `ativa` → `em_curso` → `concluida`, ou, alternativamente, `cancelada` (pelo aluno ou administrador) ou `falta` (quando o aluno não entra na sessão dentro do tempo de tolerância após o início da reserva) (Secção 4.6).

A Figura 3.2 sintetiza graficamente o modelo, apresentando o diagrama entidade-associação gerado a partir da própria base de dados, em que as ligações representadas correspondem às chaves estrangeiras declaradas no esquema. A tabela `estado_sistema` surge isolada por não ser uma entidade do domínio: guarda uma única linha, com os parâmetros configuráveis do sistema.

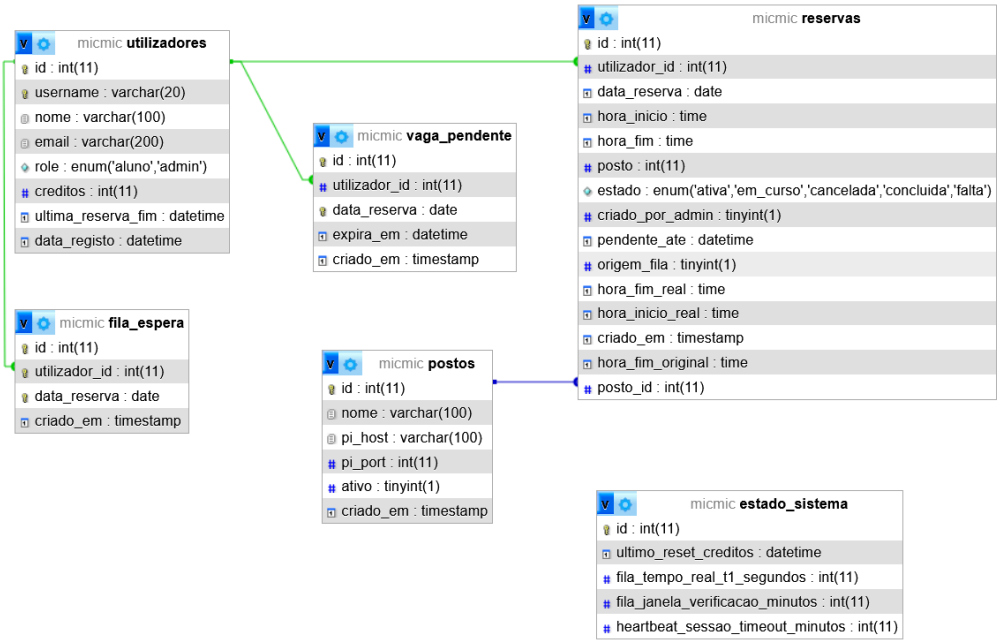


Figura 3.2: Diagrama entidade-associação da base de dados, com as chaves estrangeiras que ligam `reservas`, `fila_espera` e `vaga_pendente` à tabela `utilizadores`, e `reservas` à tabela `postos`.

3.3 Fluxo de Autenticação

A autenticação combina duas camadas distintas, seguindo os conceitos apresentados na Secção 2.3: uma camada externa, delegada por completo no SSO do DEE, e uma camada interna, de sessão local, gerida pelo próprio sistema. O fluxo completo é o seguinte:

1. O *frontend* redireciona o *browser* para a página de autenticação do DEE (`login.php`), indicando o endereço do MICMIC Lab como o URL de retorno (*requester*);

2. Após o login no DEE, o utilizador é redirecionado de volta para a aplicação, com um parâmetro `auth_jwt` no URL, que contém o JWT assinado pelo DEE;

3. O *frontend* envia esse *token*, por `POST`, para o *endpoint* `/api/auth/dee/callback` do *backend*;

4. O *backend* obtém o JWKS público do DEE (através da biblioteca `jose` [15], usando `createRemoteJWKSet`) e verifica a assinatura do JWT recebido;

5. Se a verificação for válida, o *backend* cria ou atualiza (através de uma operação *upsert*) o registo do utilizador na tabela `utilizadores`, com base no seu nome de utilizador institucional;

6. O *backend* emite então um segundo *token*, assinado localmente com um segredo próprio (algoritmo `HS256`, validade de 8 horas), que representa a sessão local do utilizador;

7. Este *token* de sessão é devolvido ao *frontend*, guardado no `localStorage`, e incluído em todos os pedidos seguintes à API, no cabeçalho `Authorization`.

O acesso às rotas protegidas da API é validado por uma função de *middleware* (`verificarToken`) que confirma a validade da sessão local antes de permitir o acesso à rota. As rotas de administração exigem uma segunda função (`verificarAdmin`) que consulta o papel (`role`) do utilizador diretamente na base de dados a cada pedido. Esta dupla verificação garante que a revogação de privilégios tenha efeitos imediatos, evitando que um *token* de sessão antigo continue a conceder acessos indevidos caso o papel do utilizador seja alterado. A atribuição do estatuto de administrador provém de uma lista estática, definida no ficheiro de configuração `config.js` do *backend*, e sincronizada com a base de dados em cada início de sessão. Neste mesmo ficheiro estão reunidos os restantes parâmetros de implantação que não constituem segredos, como os URLs de autenticação e de JWKS do DEE e o endereço público da aplicação, mantendo-os separados do código que os utiliza.

O acesso ao fluxo de vídeo em direto constitui uma exceção a este modelo. Uma vez que os elementos nativos `<video>` e `<img>` do *browser* não suportam o envio de cabeçalhos HTTP personalizados, o JWT de sessão é, neste caso específico, enviado como parâmetro de consulta (`?token=`) no próprio URL do fluxo de vídeo.

3.4 Regras de Reserva e Créditos

O acesso ao laboratório é gerido através de um orçamento semanal de créditos, expresso em minutos e associado a cada conta de aluno. Por omissão, cada aluno começa com 60 créditos, repostos semanalmente por uma tarefa em segundo plano (Secção 3.5). Para os alunos, as reservas são efetuadas em blocos predefinidos de 10, 20 ou 30 minutos, ao contrário dos administradores, que têm a possibilidade de agendar sessões de qualquer duração. Todo o cálculo temporal decorre no fuso horário de Lisboa (`Europe/Lisbon`), incluindo o tratamento de sessões que atravessem a meia-noite (transição de dia detalhada na Secção 4.3 e no Capítulo 7).

Caso não exista disponibilidade num determinado dia, o aluno pode entrar na fila de espera (tabela `fila_espera`). Para dias futuros, quando um horário é cancelado o sistema atribui uma *vaga pendente* (tabela `vaga_pendente`) ao primeiro aluno da fila, envia-lhe um email, e bloqueia esse dia exclusivamente para ele durante um período limitado. Para proteger esta atribuição ao nível da API, qualquer tentativa de reserva direta para esse dia por parte de outro utilizador é rejeitada com o código de estado HTTP `423 Locked` [16].

Este processo por email é, no entanto, demasiado demorado para vagas que surgem no *próprio* dia: o tempo entre o envio e a leitura de um email é, tipicamente, muito superior ao tempo em que uma vaga de poucos minutos permanece livre. Foi por isso acrescentado um segundo mecanismo, específico ao dia atual, que reutiliza a mesma tabela `fila_espera` mas notifica os alunos em espera através de uma ligação *Server-Sent Events* [17] mantida aberta pelo próprio *browser*, em vez de email. Sempre que uma vaga se liberta no dia atual, o sistema oferece-a aos alunos em espera de forma escalonada, por ordem de chegada à fila: o primeiro é notificado de imediato e cada um dos seguintes ao fim de um intervalo adicional, definido pelo parâmetro T_1, configurável pelo administrador. A oferta não é retirada a quem já a recebeu — ao aluno que está mais à frente na fila é dada uma vantagem de T_1 segundos sobre o colega imediatamente seguinte, e não a exclusividade da escolha — pelo que a vaga fica para quem a aceitar primeiro. Ao contrário da fila por email, aceitar uma vaga em tempo real reserva e dá imediatamente acesso ao laboratório num único passo, sem exigir uma nova visita ao calendário. Os detalhes de implementação deste mecanismo, incluindo a deteção de presença dos alunos na fila (por *heartbeat* periódico do *browser*, para retirar automaticamente quem fechou a página), são discutidos na Secção 4.10.

Uma sessão em curso pode ser terminada antecipadamente de três formas:

- **Manual (Aluno):** Através de um botão de terminar sessão mais cedo, disponível tanto na aplicação de agendamento, como na página do laboratório;
- **Administrativa:** Cancelamento por um administrador;
- **Automática:** Caso o sistema deixe de receber *heartbeats* da página do laboratório durante um período configurável (5 minutos por omissão). Este caso cobre a situação em que um aluno reserva um bloco de tempo, mas fecha a página do laboratório ou o *browser* sem terminar a sessão explicitamente, deixando o equipamento ligado e o resto do seu bloco de tempo por aproveitar por qualquer outro aluno.

Em qualquer dos cenários, o sistema executa a mesma sequência de operações: desliga o equipamento, marca a reserva como `concluida`, revoga o *token* de sessão do *Raspberry Pi* (Secção 3.7) e notifica a fila em tempo real.

O painel de controlo do aluno consulta periodicamente, através do *endpoint* `/api/verificar-acesso`, se existe uma reserva válida para o momento atual ou para os minutos seguintes. Contudo, esta verificação não altera o estado da reserva e serve apenas para atualizar o estado apresentado na interface (Secção 5.3). A transição de `ativa` para `em_curso` só ocorre quando o aluno confirma explicitamente a entrada no laboratório, através de um segundo *endpoint*, `/api/confirmar-entrada`. É também neste momento que o *backend* emite o *token* de sessão que a aplicação do *Raspberry Pi* usa para conceder o acesso físico ao equipamento, sem necessitar de consultar diretamente a base de dados a cada pedido (Secção 3.7).

3.5 Tarefas em Segundo Plano

Além de responder a pedidos HTTP, o *backend* executa um conjunto de tarefas periódicas, implementadas com recurso a temporizadores nativos do *Node.js* (`setInterval`), sem qualquer biblioteca de agendamento dedicada. Estas tarefas, executadas aproximadamente a cada minuto, são responsáveis por:

- Marcar como falta (*no-show*) reservas cuja janela de tolerância terminou sem que o utilizador tenha entrado na sessão;
- Marcar como concluídas (`concluida`) as sessões cuja janela reservada já terminou naturalmente, avisando o *Raspberry Pi* para desligar o equipamento (Secção 4.4);
- Libertação de vagas pendentes cujo prazo se tenha esgotado, promovendo o aluno seguinte na fila de espera por email;
- Terminar automaticamente sessões em curso sem *heartbeats* recentes da página do laboratório (Secção 3.4).

A fila em tempo real (Secção 3.4) tem um temporizador independente, com uma periodicidade de 30 segundos, dedicado a detetar e remover alunos que tenham deixado de dar sinal de vida enquanto esperavam por uma vaga, evitando que um aluno que tenha fechado a página sem cancelar a sua presença na fila continue, indefinidamente, a ocupar um lugar na fila e a atrasar os restantes.

A reposição semanal do orçamento de créditos (60 créditos, Secção 3.4) para os alunos é igualmente gerida em segundo plano. Para garantir a resiliência do agendamento contra reinicializações do processo ou falhas de servidor, que causariam a perda de agendamentos estritamente mantidos em memória, a data da última reposição é guardada na tabela `estado_sistema`. Periodicamente (a cada hora), o sistema calcula a segunda-feira mais recente já passada e compara-a com o registo em base de dados. Se a data guardada for anterior, o procedimento de reposição

é executado e a tabela atualizada. Isto torna o agendamento resistente a qualquer reinicialização, sem depender de um agendador externo persistente, como um *cron* do sistema operativo.

3.6 Considerações de Configuração e Segurança

A configuração do sistema divide-se por três níveis distintos, dependendo da necessidade de alteração e do ambiente de execução:

- **Variáveis de Ambiente:** Armazenadas num ficheiro `.env` excluído do controlo de versões, contêm credenciais sensíveis e parâmetros de desenvolvimento e produção. Incluem os dados de ligação à base de dados, a chave criptográfica para assinatura dos JWTs locais, o URL público da aplicação e as credenciais de envio de email. A aplicação *frontend* dispõe de um ficheiro `.env` próprio, com os endereços da API e da página de autenticação do DEE (Secção 5.1);
- **Ficheiro de Configuração:** Os parâmetros de implantação do *backend* que não constituem segredos estão reunidos num módulo `config.js` (Secção 4.1), importado pelo `server.js` no arranque. Este grupo engloba os URLs de autenticação e de JWKS do DEE e a lista de utilizadores com privilégios de administração;
- **Base de Dados:** A configuração de rede dos equipamentos do laboratório (endereço IP e porto de cada *Raspberry Pi*), inicialmente estática, é gerida diretamente na base de dados. Esta abordagem, essencial para suportar múltiplos postos de trabalho (Secção 3.8), permite aos administradores editar a infraestrutura em tempo real, sem necessidade de alterar código ou reiniciar o servidor.

3.7 Acesso ao Raspberry Pi: Token de Sessão e Canal de Revogação

O acesso físico ao equipamento não é validado, do lado do *Raspberry Pi*, através de uma consulta direta à base de dados nem de um identificador de utilizador em bruto. No momento em que o aluno confirma a entrada (Secção 3.4), o *backend* emite um *token* de sessão — um JWT autocontido e assinado com um segredo próprio (`PI_SESSION_SECRET`, distinto do usado na sessão local do aluno), contendo o identificador do utilizador, o identificador da reserva e o respetivo prazo de validade (`exp`), ajustado ao segundo exato do término da sessão. A aplicação do *Raspberry Pi* verifica a autenticidade e a validade deste *token* localmente, sem qualquer pedido de rede ao *backend*, reduzindo o acoplamento entre os dois sistemas e o número de falhas por pedido.

Dado que o JWT é autocontido, a sua validade técnica mantém-se até ao limite do prazo original, não sendo automaticamente invalidado por um cancelamento antecipado da sessão. Para eliminar esta limitação sem reintroduzir consultas constantes à base de dados, implementou-se um canal de comunicação unidirecional baseado em SSE.

Ao receber e validar o *token*, o *Raspberry Pi* subscreve este canal. Caso a sessão seja terminada antes do previsto (Secção 3.4), o *backend* emite um evento de revogação imediata. A Figura 3.3 resume esta sequência, desde a confirmação de entrada até à eventual revogação.

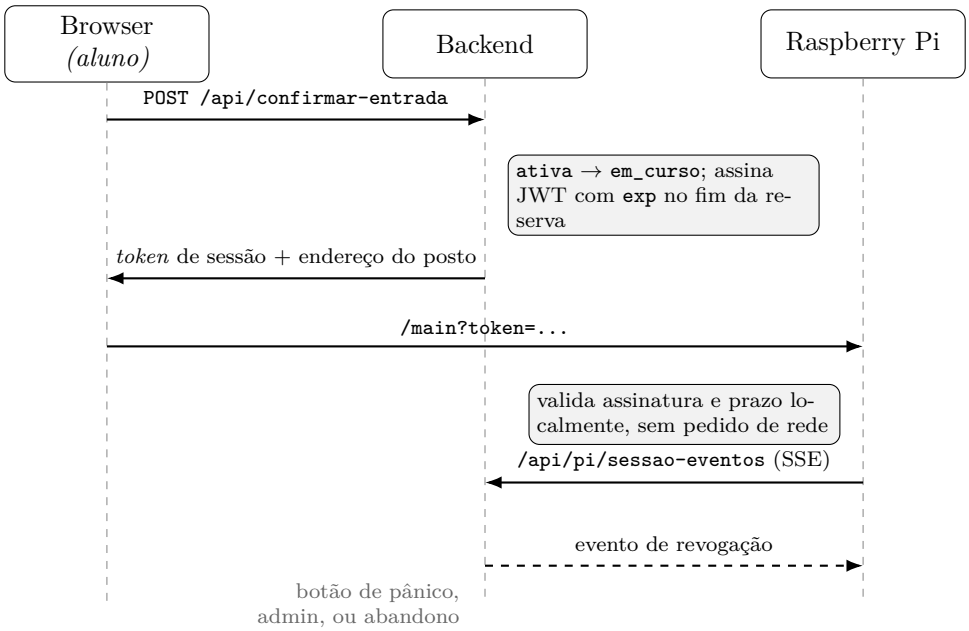


Figura 3.3: Sequência de acesso ao *Raspberry Pi*. O *token* de sessão é emitido pelo *backend* na confirmação de entrada e validado localmente pelo equipamento; o canal SSE aberto a seguir permite revogar a sessão antes do prazo de validade do *token*.

O equipamento local mantém em memória uma lista dos identificadores das sessões revogadas, e, a partir desse momento, sempre que há uma tentativa de acesso, seja à interface principal (Secção 6.1), seja ao fluxo de vídeo ou à porta série (Secção 6.3), o *Raspberry Pi* cruza o *token* apresentado com esta lista. Se o identificador constar na lista de revogação, o acesso é imediatamente bloqueado, mesmo que o prazo de validade original do *token* ainda não tenha expirado.

3.8 Suporte a Múltiplos Postos de Trabalho

O sistema foi concebido para ser capaz de suportar múltiplos postos de trabalho em simultâneo. Para acomodar esta característica, introduziu-se a entidade *posto* (tabela `postos`, Secção 3.2), em que cada posto tem o seu próprio nome, endereço

e porta, e pode ser ativado ou desativado individualmente pelo administrador sem afetar os restantes.

Cada reserva é associada a um posto específico (campo `posto_id` da tabela `reservas`). Esta atribuição é feita automaticamente pelo sistema, sendo omitida da interface do aluno uma vez que os equipamentos são equivalentes, tornando essa informação irrelevante. O sistema seleciona o primeiro posto ativo que se encontre livre durante o horário pretendido, reaproveitando as mesmas regras de sobreposição temporal já estabelecidas (Secção 4.3).

O suporte para múltiplos equipamentos altera significativamente a fórmula de cálculo da disponibilidade da fila de espera (Secção 3.4). A capacidade de atendimento do laboratório aumenta linearmente com o número de postos ativos: um bloco horário só é considerado esgotado quando todos os postos em funcionamento possuem uma reserva sobreposta nesse período. Como a atribuição física é mantida como um detalhe interno da API, a experiência do utilizador permanece inalterada.

Na interface de administrador, o painel "Posto ao Vivo" (Secção 5.4), apresenta um seletor que permite inspecionar a disponibilidade e a sessão em curso de cada unidade individualmente. Ao nível do *backend*, todas as rotas de encaminhamento do proxy para o *Raspberry Pi* — transmissão de vídeo, estado e comandos de *hardware* — passam a resolver dinamicamente o IP de destino com base no posto associado ao pedido.

Capítulo 4

Implementação do Backend

Este capítulo descreve a implementação da API *backend*, o componente central do sistema, responsável pela autenticação, pela lógica de reservas e créditos, pela execução de tarefas em segundo plano e pela comunicação com o *Raspberry Pi*.

É dado particular destaque aos aspetos que se revelaram mais delicados. O primeiro é o tratamento de reservas que atravessam a meia-noite, que se manifesta em quatro pontos distintos do sistema — a deteção de conflitos de horário, a deteção de faltas, o cálculo do encerramento antecipado e a janela de procura da fila em tempo real — e que exige, em todos eles, abandonar a comparação de horas em favor de instantes absolutos. O segundo é a garantia de que o equipamento é sempre desligado no final de uma sessão, independentemente da forma como esta termina. Descreve-se ainda a política de créditos associada ao botão de terminar sessão antecipadamente, o mecanismo que permite ao administrador operar diretamente o *hardware* do laboratório, e a emissão do *token* de sessão que autentica o aluno junto do *Raspberry Pi*.

Adicionalmente foram implementadas as seguintes funcionalidades: uma fila de espera em tempo real para vagas do próprio dia, com deteção de presença por *heartbeat*; a deteção e terminação automática de sessões abandonadas pelo aluno sem aviso explícito; e o suporte a múltiplos postos de trabalho intercambiáveis, com atribuição automática e sem qualquer alteração visível à experiência do aluno.

4.1 Estrutura e Tecnologias

O *backend* está implementado num único ficheiro *Node.js* (`server.js`), com cerca de 3200 linhas, recorrendo à *framework Express* [13], na sua versão 5. Este único processo acumula três responsabilidades: serve os ficheiros estáticos da aplicação *React* já compilada (Capítulo 5), disponibiliza a API implementada no *backend* sob o prefixo `/api` e atua como cliente HTTP do *Raspberry Pi*, para verificação de acessos e o envio de ordens de encerramento do equipamento (Secção 4.4).

A opção por concentrar a arquitetura num único ficheiro, em vez de a distribuir por múltiplos módulos, foi uma decisão prática, adequada à dimensão do projeto. No entanto, com a evolução do sistema, poderá tornar-se necessário fazer a divisão do ficheiro em módulos mais pequenos (autenticação, reservas, administração), como referido no Capítulo 8.

A esta opção escapam os parâmetros de configuração. Os valores que dependem da implantação e não constituem segredos — a lista de administradores, os endereços de autenticação do DEE e o endereço público da aplicação — estão reunidos num módulo `config.js` (Listagem 4.1), importado pelo `server.js` no arranque. Mudar de servidor ou de laboratório resume-se assim a editar este ficheiro, em vez de procurar constantes dispersas por milhares de linhas. Os segredos propriamente ditos — credenciais da base de dados, chave de assinatura dos JWTs e credenciais de envio de email — permanecem no ficheiro `.env`, excluído do controlo de versões (Secção 3.6).

```
1  const DEE_BASE = 'https://www.ave.dee.isep.ipp.pt/DEE';
2
3  module.exports = {
4      // Usernames institucionais com privilégios de administração
5      ADMINS_UC: ['2250002', 'jes'],
6
7      // Autenticação do DEE
8      DEE_JWKS_URL: `${DEE_BASE}/auth/.well-known/jwks.json`,
9      DEE_LOGIN_URL: `${DEE_BASE}/auth/login.php`,
10
11     // Endereço público da aplicação. O .env tem precedência.
12     APP_URL: process.env.APP_URL || `${DEE_BASE}/micmic-lab`
13 };
```

Listagem 4.1: Módulo de configuração do *backend*, que reúne os parâmetros de implantação que não constituem segredos.

4.2 Principais Rotas da API

As Tabelas 4.1 e 4.2 resumem as operações mais relevantes da API implementada no *backend*, separadas conforme sejam consumidas pela aplicação *frontend* ou pela aplicação do laboratório, no *Raspberry Pi*.

Tabela 4.1: Principais operações da API consumidas pela aplicação *frontend*.

Método	Rota	Descrição
`POST`	`/api/auth/dee/callback`	Recebe o JWT do DEE, valida-o e emite a sessão local (Secção 3.3).
`POST`	`/api/reservar`	Cria uma nova reserva, aplicando as regras de créditos, duração e deteção de conflitos (Secção 4.3).
`POST`	`/api/verificar-acesso`	Confirma, para o utilizador autenticado, se existe uma reserva válida para o momento atual ou para os minutos seguintes, sendo consultada pelo painel de controlo do aluno (Secção 5.3).
`POST`	`/api/confirmar-entrada`	Confirma a entrada efetiva do aluno no laboratório, transitando a reserva de `ativa` para `em_curso` e emitindo o *token* de sessão do *Raspberry Pi* (Secção 4.8).
`POST`	`/api/cancelar-antecipado`	Termina uma sessão em curso antes do previsto ("botão de pânico"), desligando o equipamento; não devolve créditos (Secção 4.4).
`POST`	`/api/admin/lab-switch` `/api/admin/lab-dac` `/api/admin/lab-serial`	Permitem ao administrador operar diretamente o *hardware* do laboratório (Secção 4.7).
`GET`	`/api/minhas-reservas`	Lista as reservas do utilizador autenticado.
`GET`	`/api/fila-tempo-real/eventos`	Canal de eventos (SSE) da fila em tempo real do dia atual (Secção 4.10).
`POST`	`/api/fila-tempo-real/aceitar`	Aceita a vaga oferecida, reservando e dando acesso ao laboratório num único passo (Secção 4.10).
`GET/POST/PUT/DELETE`	`/api/admin/*`	Conjunto de rotas de administração (gestão de utilizadores, reservas, configurações e postos de trabalho), protegidas pela função `verificarAdmin`.

As operações da Tabela 4.1 estão protegidas pela função de *middleware* `verificarToken`, apresentada na Secção 3.3, com duas exceções: a rota

`/api/auth/dee/callback`, que tem de estar acessível antes de existir qualquer sessão local; e as rotas consumidas diretamente por elementos do *browser* que não permitem cabeçalhos HTTP personalizados (`/api/admin/lab-video` e `/api/fila-tempo-real/eventos`), que recebem o JWT de sessão como parâmetro de consulta.

Tabela 4.2: Operações da API consumidas pela aplicação do laboratório, no *Raspberry Pi*.

Método	**Rota**	**Descrição**
`GET`	`/api/pi/sessao-eventos`	Canal de eventos (SSE) a que o *Raspberry Pi* se liga para receber uma eventual revogação da sessão em curso (Secção 4.8).
`POST`	`/api/sessao/heartbeat`	Sinal de vida enviado periodicamente pela página do laboratório, enquanto a sessão está em curso (Secção 4.11).
`POST`	`/api/sessao/terminar`	Encerramento antecipado a pedido do aluno, a partir da própria página do laboratório (Secção 4.11).

As operações da Tabela 4.2 não são invocadas pelo *browser* através da aplicação de agendamento, mas pela própria aplicação do laboratório, que não tem acesso à sessão local do aluno. São por isso autenticadas pelo *token* de sessão emitido no momento da entrada (Secção 3.7), que o *Raspberry Pi* apresenta em cada pedido.

4.3 Deteção de Conflitos de Horário

A rotina que determina se um novo pedido de reserva entra em conflito com reservas já existentes é a parte mais delicada do sistema, devido à necessidade de validar transições de dia que atravessam a meia-noite. A verificação considera três cenários distintos, sintetizados no Algoritmo 4.1:

1. **Sobreposição direta no mesmo dia:** Interseção temporal de sessões no mesmo dia da reserva;
2. **Reserva anterior estendida:** Existência de uma reserva iniciada no dia anterior cuja janela ultrapassa a meia-noite e avança pelo dia da nova reserva;
3. **Nova reserva estendida:** Situação em que a *nova* reserva atravessa a meia-noite, podendo colidir com reservas agendadas para o dia seguinte.

A Listagem 4.2 apresenta a instrução SQL que implementa este algoritmo diretamente na base de dados, evitando assim a transferência desnecessária de registos para memória.

Require: data, horaInicio, horaFim, cruzouMeiaNoite
1 conflito ← **falso**
2 **for all** reserva r com estado `ativa` ou `em_curso` **do**
3 **if** r.data = data **e** r.horaInicio < horaFim **e** r.horaFim > horaInicio **then**
4 conflito ← **verdadeiro** ▷ Cenário 1: sobreposição no mesmo dia
5 **else if** r.data = data − 1 dia **e** r.horaFim < r.horaInicio **e** horaInicio < r.horaFim **then**
6 conflito ← **verdadeiro** ▷ Cenário 2: reserva existente vinda do dia anterior
7 **else if** cruzouMeiaNoite **e** r.data = data + 1 dia **e** horaFim > r.horaInicio **then**
8 conflito ← **verdadeiro** ▷ Cenário 3: nova reserva estende-se para o dia seguinte
9 **end if**
10 **end for**
11 **return** conflito

Algoritmo 4.1: Deteção de conflitos de horário no momento da reserva

```
1  SELECT id FROM reservas
2  WHERE estado IN ('ativa', 'em_curso')
3  AND (
4      (data_reserva = ? AND hora_inicio < ? AND hora_fim > ?)
5      OR (data_reserva = DATE_SUB(?, INTERVAL 1 DAY)
6          AND hora_fim < hora_inicio
7          AND TIME_TO_SEC(?) < TIME_TO_SEC(hora_fim))
8      OR (? = 1
9          AND data_reserva = DATE_ADD(?, INTERVAL 1 DAY)
10         AND ? > hora_inicio
11         AND TIME_TO_SEC(?) > TIME_TO_SEC(hora_inicio))
12 )
```

Listagem 4.2: Instrução SQL de deteção de conflitos de horário, com o parâmetro `cruzouMeiaNoite` ligado ao Cenário 3.

A inclusão do parâmetro `cruzouMeiaNoite` na verificação do Cenário 3 (linhas 8–11 da Listagem 4.2) resolve uma limitação crítica da lógica de agendamento. Sem esta condição, o Cenário 3 seria avaliado incondicionalmente, mesmo para reservas contidas num único dia. Isso faria com que uma nova reserva pudesse ser incorretamente rejeitada por conflito com uma reserva do dia seguinte, caso a hora de início dessa reserva futura fosse anterior à hora de fim da reserva atual. A solução consistiu em vincular este ramo à variável `cruzouMeiaNoite`, calculada no *backend* através da comparação entre os horários de início e fim da reserva, e injetada como parâmetro binário na consulta SQL (`cruzouMeiaNoite ? 1 : 0`).

4.4 Encerramento Seguro do Equipamento

Sempre que uma sessão termina — quer de forma natural, pela tarefa em segundo plano que fecha as reservas (Secção 3.5), quer de forma antecipada, por iniciativa do utilizador através do botão de terminar sessão mais cedo —, o sistema assegura a desativação física do *hardware* laboratorial (interruptores e conversor digital-analógico) associado a essa reserva, através do *endpoint* `/api/desligar-tudo` no *Raspberry Pi*. A Listagem 4.3 apresenta a função auxiliar desenvolvida para centralizar esta comunicação.

```
function desligarRaspberryPi(utilizadorId, postoId, callback) {
    if (!utilizadorId) return callback();
    const posto = POSTOS_ATIVOS.find(p => p.id === postoId) ||
    POSTOS_ATIVOS[0];
    if (!posto) return callback();
    const piReq = http.request({
        host: posto.pi_host,
        port: posto.pi_port,
        path: `/api/desligar-tudo?utilizador_id=${
    encodeURIComponent(utilizadorId)}`,
        method: 'GET',
        timeout: 5000
    }, (piRes) => {
        piRes.on('data', () => {});
        piRes.on('end', () => callback());
    });
    piReq.on('error', (e) => {
        console.error('Falha ao contactar Pi para desligar:', e.
    message);
        callback();
    });
    piReq.on('timeout', () => { piReq.destroy(); });
    piReq.end();
}
```

Listagem 4.3: Função auxiliar responsável por avisar o Raspberry Pi para desligar o equipamento antes de uma reserva ser fechada.

A centralização deste mecanismo resolve um problema clássico de dessincronização entre o estado mantido na base de dados e o estado físico do *hardware*. Uma sessão pode terminar por várias vias — pela tarefa automática de fim de sessão (Secção 3.5), pelo botão de encerramento antecipado ou por ação do administrador — e basta que uma delas não notifique o *Raspberry Pi* para que a reserva fique encerrada no sistema com os interruptores da bancada fisicamente ligados. Para garantir que nenhum equipamento permanece ativo após o encerramento da sessão, o *backend*

está estruturado de forma a que a função `desligarRaspberryPi` seja obrigatoriamente executada antes de qualquer alteração ao estado da reserva na base de dados, como exemplificado na Listagem 4.4 para o caso de encerramento de sessão mais cedo.

```
app.post('/api/cancelar-antecipado', verificarToken, (req, res) => {
    // ... obter 'reserva' correspondente ao pedido ...
    desligarRaspberryPi(reserva.utilizador_id, reserva.posto_id, () => {
        // UPDATE reservas SET estado = 'concluida', hora_fim_real = NOW() ...
        res.json({ message: 'Sessão encerrada mais cedo. Os créditos não são devolvidos.' });
    });
});
```

Listagem 4.4: Rota de cancelamento antecipado, agora a avisar sempre o Raspberry Pi antes de fechar a reserva na base de dados (excerto simplificado).

Esta discrepância ilustra um tipo de problema recorrente em sistemas que combinam estado em base de dados com estado físico de *hardware*: as duas partes podem, com facilidade, divergir uma da outra se não existir um ponto de sincronização entre elas.

O encerramento antecipado não devolve créditos ao aluno. A reserva é fechada como `concluida`, e não `cancelada`, para que a sessão continue a contar nas estatísticas do painel de administração (Secção 5.4), mas o tempo não utilizado não regressa ao saldo. Esta é a única exceção deliberada ao princípio de proporcionalidade seguido pelo resto da política de créditos do sistema: pretende-se desincentivar reservas feitas com folga e terminadas sistematicamente mais cedo, que ocupam blocos que outros alunos poderiam ter reservado.

O mesmo cuidado com a meia-noite aplica-se aqui. Ao arredondar a hora de fim para o bloco de 10 minutos seguinte, uma sessão terminada às 23:55 passa a terminar às 00:00 do dia seguinte — pelo que a decisão sobre a que dia pertence o fim da reserva tem de assentar na hora real, e não na arredondada.

4.5 Envio de Notificações por Email

O envio de emails, usado para notificar um aluno de que uma vaga se libertou na fila de espera, é implementado num módulo isolado (`enviarEmail.js`). A implementação recorre à biblioteca `Nodemailer` [18] ligada a um serviço Gmail, utilizando credenciais de autenticação armazenadas em variáveis de ambiente.

O corpo da mensagem, apresentado na Figura 4.1, indica o dia atribuído e a hora-limite até à qual a vaga se mantém reservada, e inclui uma ligação que encaminha o aluno diretamente para o sistema já com esse dia pré-selecionado no calendário. O horário concreto dentro do dia é escolhido pelo próprio aluno, uma vez que o que lhe é atribuído é o dia e não um bloco específico.



Figura 4.1: Email de notificação enviado ao primeiro aluno da fila de espera quando uma vaga é libertada, com a hora-limite para a marcação e a ligação direta para o dia atribuído.

Este módulo foi projetado para capturar e tratar internamente qualquer erro. Qualquer falha no envio é registada em *log* e a função retorna o valor `false`, em vez de interromper o fluxo do pedido HTTP que a invocou. Esta opção reflete a natureza não crítica desta funcionalidade: uma falha temporária no envio de um email não impede o funcionamento do resto do sistema nem prejudica a operação principal, nomeadamente a atribuição efetiva da vaga pendente.

4.6 Estado de Falta e Deteção de Não-Comparência

A coluna `estado` da tabela `reservas` (Secção 3.2) distingue a falta do cancelamento voluntário através de dois valores próprios. A distinção é necessária porque os dois casos são diferentes do ponto de vista da gestão do laboratório: um cancelamento prévio liberta o horário para os restantes alunos, ao passo que uma não-comparência deixa o equipamento reservado e por utilizar. Só com estados distintos é possível contabilizar faltas por aluno — através de uma consulta estatística dedicada, apresentada no painel de administração (Secção 5.4) — e viabilizar futuros mecanismos de penalização por faltas recorrentes (Capítulo 8).

A deteção de não-comparências é um dos pontos em que o tratamento das reservas que atravessam a meia-noite se torna crítico. Avaliar apenas os minutos decorridos desde a meia-noite, convertendo os campos `hora_inicio` e `hora_fim` em inteiros, ignora as reservas com transição de dia e torna impossível assinalar a falta numa sessão que termina no dia seguinte àquele em que começou.

A tarefa de deteção de faltas, apresentada na Listagem 4.5, compara por isso instâncias do objeto `Date`, construídas a partir da data e da hora específicas de cada reserva, em vez de inteiros, e consulta tanto o dia atual como o dia anterior, replicando o padrão de dupla consulta adotado na deteção de conflitos de horário (Secção 4.3).

```
setInterval(() => {
    const agora = new Date(new Date().toLocaleString('en-US', { timeZone: 'Europe/Lisbon' }));
    const dataHoje = formatarData(agora);
    const dataOntem = formatarData(new Date(agora).setDate(agora.getDate() - 1));

    db.query(
        `SELECT id, hora_inicio, hora_fim, data_reserva FROM reservas
         WHERE estado = 'ativa' AND (data_reserva = ? OR data_reserva = ?)`,
        [dataHoje, dataOntem],
        (err, reservas) => {
            reservas.forEach((r) => {
                const inicio = construirData(r.data_reserva, r.hora_inicio);
                const fim = construirData(r.data_reserva, r.hora_fim);
                if (fim <= inicio) fim.setDate(fim.getDate() + 1); // cruza a meia-noite

                const passaramDezMinutos = agora > new Date(inicio.getTime() + 10 * 60000);
                const aindaNoIntervalo = agora < fim;
                if (passaramDezMinutos && aindaNoIntervalo) {
                    db.query("UPDATE reservas SET estado = 'falta' WHERE id = ?", [r.id]);
                }
            });
        }
    );
}, 60000);
```

Listagem 4.5: Tarefa de deteção de faltas, com o tratamento das reservas que atravessam a meia-noite (excerto simplificado).

Fica registada uma limitação aceite, e deliberadamente não corrigida, desta janela de tolerância de dez minutos: em reservas com a duração mínima do sistema (10 minutos, Secção 3.4), a condição `passaramDezMinutos` nunca se torna verdadeira antes de a própria reserva já ter terminado, tornando matematicamente impossível marcar como falta uma reserva desta duração. Considerou-se mais importante manter dez minutos de tolerância, um valor razoável antes de considerar uma falta, do que garantir a deteção também neste caso-limite, que corresponde a uma pequena parte do total de reservas efetuadas.

4.7 Controlo de Hardware pelo Administrador

Para permitir a intervenção remota por parte dos administradores foi desenvolvido um painel que permite aos docentes acompanhar, diagnosticar ou intervir diretamente numa sessão em tempo real, sem necessidade de deslocação física ao laboratório. Este módulo permite a operação direta do *hardware* (controlo de interruptores, conversor digital-analógico e envio de texto pela porta série). Ao contrário de todas as outras rotas apresentadas neste capítulo, estas ações não estão associadas a qualquer reserva em vigor, podendo ser invocadas independentemente da existência de uma sessão ativa.

Esta especificidade inviabilizou a utilização do mecanismo convencional de validação de reservas implementado no *Raspberry Pi* (Secção 6.1), pensado para validar o acesso de um aluno a uma sessão concreta, e não para ações administrativas. Introduziu-se, em alternativa, um mecanismo de autenticação baseado num segredo partilhado entre o *backend* e o *Raspberry Pi* (`PI_ADMIN_SECRET`). Este segredo, gerado aleatoriamente e guardado apenas nas variáveis de ambiente de cada um dos dois lados, nunca é transmitido ao *frontend*.

Conforme ilustrado na Listagem 4.6, os pedidos de controlo administrativo são primeiramente sujeitos às funções de validação de sessão (`verificarToken`) e de privilégios de administração (`verificarAdmin`) (Secção 3.3). Após a validação no *backend*, o pedido é reencaminhado pelo proxy para o *Raspberry Pi*, injetando o segredo no cabeçalho HTTP `X-Admin-Secret`. A aplicação do *Raspberry Pi* confirma esta credencial antes de executar qualquer comando sobre o *hardware* (Secção 6.4).

```
1 function proxyComandoParaPi(posto, caminho, corpo, res) {
2     const dados = JSON.stringify(corpo);
3     const piReq = http.request({
4         host: posto.pi_host, port: posto.pi_port,
```

```
5           path: caminho, method: 'POST', timeout: 5000,
6           headers: {
7               'Content-Type': 'application/json',
8               'Content-Length': Buffer.byteLength(dados),
9               'X-Admin-Secret': PI_ADMIN_SECRET
10          }
11      }, (piRes) => {
12          let body = '';
13          piRes.on('data', c => body += c);
14          piRes.on('end', () => res.status(piRes.statusCode).send(
        body));
15      });
16      piReq.on('error', () => res.status(502).json({ message: '
        Laboratório offline ou inacessível' }));
17      piReq.write(dados);
18      piReq.end();
19  }
20
21  app.post('/api/admin/lab-switch', verificarToken, verificarAdmin,
        (req, res) => {
22      const posto = obterPostoDoPedido(req, res);  // resolve o
        posto indicado no pedido
23      if (!posto) return;
24      proxyComandoParaPi(posto, '/api/admin/switch', { nome: req.
        body.nome }, res);
25  });
```

Listagem 4.6: Função auxiliar que reencaminha um comando de hardware para o Raspberry Pi, autenticado pelo segredo partilhado entre backend e Pi.

No *Raspberry Pi*, as três novas rotas correspondentes (`/api/admin/switch`, `/api/admin/dac` e `/api/admin/serial`) reutilizam a mesma lógica de *General Purpose Input/Output* (GPIO), de conversão digital-analógico e comunicação por porta série (Secção 6.4). A invocação destas rotinas passa a ser efetuada mediante pedidos com dados em formato JSON autenticados pelo cabeçalho de segurança, promovendo a reutilização da lógica de controlo do *hardware* e evitando a duplicação de código na aplicação do *Raspberry Pi*.

4.8 Emissão do Token de Sessão e Canal de Revogação

Em conformidade com a arquitetura descrita na Secção 3.7, o *endpoint* `/api/confirmar-entrada` passou a emitir, além de transitar o estado da reserva, um *token* de sessão assinado com um segredo dedicado (`PI_SESSION_SECRET`), cujo prazo de validade é calculado a partir da hora de fim da reserva, como mostra a Listagem 4.7.

```
const horaFimObj = new Date(ano, mes - 1, dia, hFim, mFim, 0);
if (horaFimObj <= horaInicioObj) horaFimObj.setDate(horaFimObj.
    getDate() + 1);

const tokenSessao = jwt.sign(
    { utilizador_id, reserva_id: reserva.id, exp: Math.floor(
    horaFimObj.getTime() / 1000) },
    PI_SESSION_SECRET
);
```

Listagem 4.7: Cálculo do prazo de validade do token de sessão, com o mesmo tratamento de reservas que atravessam a meia-noite usado no resto do sistema (excerto simplificado).

Para o canal de revogação, o *backend* mantém um mapa em memória de sessões ativas (identificador da reserva → ligação SSE aberta), e expõe um *endpoint* dedicado ao qual o próprio *Raspberry Pi* se liga assim que recebe o *token*, utilizando-o como prova de autenticidade da sessão.

```
app.get('/api/pi/sessao-eventos', (req, res) => {
    const payload = jwt.verify(req.query.token, PI_SESSION_SECRET)
    ;
    res.setHeader('Content-Type', 'text/event-stream');
    res.flushHeaders();
    sessoesAtivas.set(payload.reserva_id, res);

    const heartbeat = setInterval(() => res.write(': heartbeat\n\n
    '), 15000);
    req.on('close', () => {
        clearInterval(heartbeat);
        sessoesAtivas.delete(payload.reserva_id);
    });
});
```

Listagem 4.8: Endpoint de eventos de sessão: o Raspberry Pi liga-se aqui, autenticado pelo próprio token, e a ligação fica registada para receber uma eventual revogação.

A função `revogarSessao(reservaId)` envia um evento `revoke` através da ligação correspondente, se esta existir. É invocada sempre que uma sessão deixa de ser válida antes da expiração do *token*, a partir de três pontos do código: o encerramento antecipado, concentrado na função `terminarSessaoAntecipadamente` e partilhado pelo botão do aluno, pelo botão da página do laboratório e pela deteção de abandono (Secção 4.11); o cancelamento de uma sessão em curso por um administrador; e a tarefa de fecho automático de sessões (Secção 3.5), onde serve sobretudo para

encerrar formalmente a ligação SSE e libertar a estrutura de dados em memória, dado que o *token* expira nesse mesmo instante.

Durante a fase de testes, este canal evidenciou um comportamento imprevisto no desenho inicial: a ligação SSE caía autonomamente após cerca de um minuto sem qualquer ocorrência de revogação, originando o erro `InvalidChunkLength` do lado do *Raspberry Pi*. A causa foi o *proxy* reverso do ISEP (Secção 3.1), que encerra ligações HTTP inativas por períodos prolongados. Como a ligação em causa apenas transmitia dados no momento de uma revogação, uma sessão sem incidentes permanecia sem tráfego.

A resolução compreendeu duas etapas: o *backend* passou a emitir periodicamente um comentário nulo de *heartbeat* (Listagem 4.8, com intervalo de 15 segundos) para manter a ligação ativa no *proxy*; por sua vez, o *Raspberry Pi* foi programado para reconectar-se automaticamente caso a ligação caia por qualquer motivo, desde que o *token* permaneça válido (Secção 6.1).

4.9 Persistência do Reset Semanal de Créditos

Em conformidade com a limitação descrita na Secção 3.5, a tabela `estado_sistema`, constituída por uma única linha e criada automaticamente, tal como as restantes tabelas do sistema (Secção 3.2), guarda a data da segunda-feira mais recente em que o reset de créditos foi efetivamente executado. A Listagem 4.9 apresenta a verificação periódica correspondente.

```
1  function calcularSegundaMaisRecente(agora) {
2      const diasDesdeSegunda = (agora.getDay() + 6) % 7; // segunda
       =0, terça=1, ...
3      const segunda = new Date(agora);
4      segunda.setDate(agora.getDate() - diasDesdeSegunda);
5      segunda.setHours(0, 0, 0, 0);
6      return segunda;
7  }
8
9  function verificarResetSemanal() {
10     const agora = new Date(new Date().toLocaleString('en-US', {
       timeZone: 'Europe/Lisbon' }));
11     const segundaMaisRecente = calcularSegundaMaisRecente(agora);
12     db.query('SELECT ultimo_reset_creditos FROM estado_sistema
       WHERE id = 1', (err, results) => {
13         const ultimoReset = results[0]?.ultimo_reset_creditos;
14         if (!ultimoReset || new Date(ultimoReset) <
       segundaMaisRecente) {
15             executarReset(segundaMaisRecente);
16         }
```

```
    });
}
```

Listagem 4.9: Verificação periódica do reset semanal, comparando a segunda-feira mais recente já passada com a data do último reset guardada na base de dados (excerto simplificado).

Esta verificação é executada a cada hora e também no arranque do processo, cobrindo tanto o cenário normal (em que a tarefa deteta a segunda-feira no momento previsto) como a situação em que o serviço esteve indisponível durante a transição: ao reiniciar, o sistema deteta o atraso e executa o reset pendente, evitando que este seja omitido até à segunda-feira subsequente.

4.10 Fila de Espera em Tempo Real

Correspondendo à arquitetura descrita na Secção 3.4, foi acrescentado um segundo mecanismo de fila de espera, dedicado exclusivamente a vagas que se libertam no próprio dia, desenhado em conjunto com o orientador do projeto. Ao contrário da fila por email, que trata um dia inteiro como uma unidade "cheio ou não cheio", este mecanismo tem de responder à pergunta mais fina de *quando*, dentro do próprio dia, já não sobra nada aproveitável em breve.

Essa decisão está a cargo da função `naoHaVagaParaEntrarNaFila`: para o dia atual, um aluno só é encaminhado para a fila em tempo real se não existir nenhum bloco livre dentro de uma janela de verificação configurável pelo administrador (60 minutos por omissão) a partir do momento presente — se já houver algo livre mais tarde dentro dessa janela, reservar diretamente é preferível a esperar por uma vaga imprevisível. Para dias futuros, mantém-se o critério original de dia completamente esgotado (Secção 3.4). A Listagem 4.10 apresenta este critério.

Esta janela não é interrompida à meia-noite: às 23:55, por exemplo, com o valor por omissão, o sistema procura vagas até às 00:55 do dia seguinte, e a vaga oferecida pode igualmente atravessar a meia-noite, como qualquer outra reserva do sistema.

```
function naoHaVagaParaEntrarNaFila(data, ocupadosSet,
    janelaMinutos) {
    if (data === dataHojeLisboa()) {
        return proximaVagaDentroDaJanela(ocupadosSet,
    janelaMinutos) === null;
    }
    return ocupadosSet.size >= slotsTotalDoDia(data);
}
```

Listagem 4.10: Critério que decide se um aluno deve ser encaminhado para a fila de espera: janela de verificação para hoje, dia completo para dias futuros (excerto simplificado).

Quando uma vaga se liberta hoje — por cancelamento, por fim de sessão antecipado (Secção 4.11) ou natural — a função `processarProximoDaFila` distingue o dia da vaga: se for no dia atual, delega em `processarFilaTempoReal`; caso contrário, mantém o fluxo original por email (Secção 3.4). A oferta aos alunos em fila não é sequencial e bloqueante (esperar a resposta de um antes de sequer notificar o seguinte, como acontecia na fila por email), mas sim progressiva. Todos os candidatos com créditos são agendados de uma vez, cada um com um atraso proporcional à sua posição: o aluno na posição n da fila recebe a oferta ao fim de $T_1 \times (n-1)$ segundos — o primeiro recebe-a de imediato, o segundo passados T_1 segundos, o terceiro passados $2 \times T_1$, e assim sucessivamente — sem retirar a oferta a quem já a recebeu. Isto significa que, em qualquer instante, pode haver mais do que um aluno com uma oferta ativa em simultâneo; ganha a vaga quem a aceitar primeiro. A Figura 4.2 ilustra esta sobreposição progressiva.

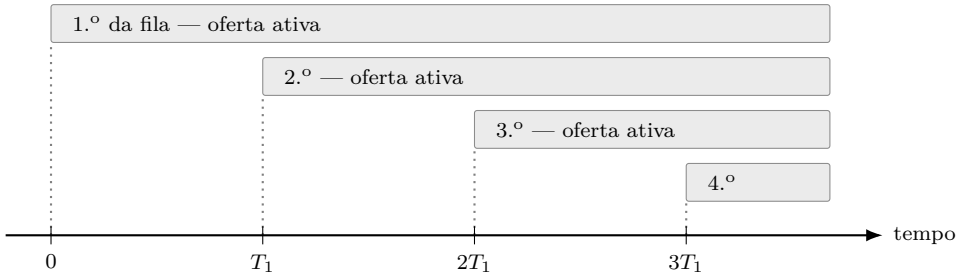


Figura 4.2: Revelação progressiva de uma vaga libertada. O aluno na posição n da fila recebe a oferta ao fim de $T_1 \times (n-1)$ segundos e mantém-na até alguém aceitar, pelo que as ofertas se acumulam em vez de se substituírem: quem está mais à frente na fila tem uma vantagem de T_1 segundos sobre o seguinte, e não a exclusividade da escolha.

O Algoritmo 4.2 descreve este agendamento, incluindo a verificação, feita no momento de enviar cada oferta, de que ainda resta uma janela livre aproveitável.

O cálculo de "janela livre agora" (`calcularJanelaLivreAgora`) é feito no momento exato da oferta, e não no momento em que a ronda foi agendada — o tempo passa entre uma e outra, e outra pessoa pode entretanto ter reservado o bloco em causa. A janela devolvida é sempre o que sobra realmente livre a partir de agora, arredondado ao bloco de 10 minutos em curso, até ao primeiro bloco ocupado ou até um máximo de 30 minutos — pelo que a sua duração raramente é, ela própria, um múltiplo redondo de 10 minutos (por exemplo, 8 ou 22 minutos), refletindo deliberadamente "o que sobrou", e não uma marcação a uma hora à escolha do

```
Require: candidatos (lista ordenada por chegada à fila, só com créditos > 0), T_1 (segundos)
for all candidato c de índice i (a partir de 0) em candidatos do
    atraso ← T_1 × i segundos
    agendar OfertarVaga(c) para daqui a atraso
end for

procedure OfertarVaga(utilizadorId)
    if oferta já enviada a utilizadorId nesta ronda then
        return      ▷ ex: uma recusa explícita já acelerou a oferta a esta pessoa
    end if
    marcar oferta enviada a utilizadorId
    janela ← CalcularJanelaLivreAgora
    if janela = nulo then
        return      ▷ já não sobra nada aproveitável — não vale a pena notificar
    end if
    enviar evento SSE vaga-disponivel a utilizadorId, com janela
end procedure
```

Algoritmo 4.2: Agendamento em cascata das ofertas da fila em tempo real

aluno. Aceitar esta oferta (`POST /api/fila-tempo-real/aceitar`) cria e confirma a reserva num único passo — sem passar pelo calendário — atribui-lhe automaticamente um posto livre (Secção 3.8) e devolve de imediato o *token* de sessão do *Raspberry Pi* (Secção 4.8), dando ao aluno acesso ao laboratório de forma tão direta como a de quem já tinha uma reserva agendada.

A presença de cada aluno na própria fila (distinta da deteção de abandono de uma sessão já em curso, Secção 4.11) é também acompanhada por *heartbeat*: o *browser* do aluno mantém uma ligação SSE aberta (`GET /api/fila-tempo-real/eventos`) enquanto aguarda na fila, e envia periodicamente um sinal de vida (`POST /api/fila-tempo-real/heartbeat`); uma tarefa a cada 30 segundos remove da fila quem não dá esse sinal há mais de 90 segundos (três vezes o intervalo esperado), libertando de imediato a posição para os restantes. Um cuidado deliberado nesta limpeza: o fecho da própria ligação SSE (evento `close`, disparado quase de imediato quando o aluno muda de aba ou tem uma quebra de rede momentânea) apaga apenas o registo dessa ligação, nunca o último *heartbeat* conhecido — só a ausência prolongada de novos *heartbeats* é que retira mesmo o aluno da fila. Sem esta distinção, o *EventSource* do *browser*, que se religa sozinho após uma quebra temporária, nunca teria oportunidade de o fazer, porque o aluno já teria sido removido da fila no primeiro sinal de instabilidade de rede.

4.11 Deteção de Abandono de Sessão

Para evitar que uma sessão permaneça ocupada quando o aluno deixa de utilizar o laboratório sem a encerrar formalmente, foi implementado um mecanismo de deteção de abandono baseado em *heartbeats*. Por exemplo, um aluno pode reservar um bloco de 30 minutos, utilizar o equipamento durante 5 minutos e fechar o *browser* sem clicar no botão de terminar sessão. Sem este mecanismo de verificação, o equipamento permaneceria ativo e o restante tempo reservado ficaria inacessível a outros alunos.

A solução adotada foi fazer este sinal de vida (*heartbeat*) partir da própria página do laboratório servida pelo *Raspberry Pi* (`main.html`), e não da aplicação de agendamento: apenas a primeira reflete a utilização ativa do laboratório, ao contrário da segunda que indica somente que o aluno mantém um separador do agendamento aberto. A página do *Raspberry Pi* envia um *heartbeat* (`POST /api/sessao/heartbeat`, autenticado pelo próprio *token* de sessão, Secção 3.7) a cada 30 segundos enquanto a sessão estiver `em_curso`. O *backend* guarda apenas o instante do último sinal recebido, por reserva, num mapa em memória. A Listagem 4.11 apresenta a tarefa que deteta a ausência prolongada desse sinal.

```
setInterval(() => {
    obterHeartbeatTimeoutMinutos((errCfg, timeoutMinutos) => {
        const limiteMs = timeoutMinutos * 60 * 1000;
        const agora = Date.now();
        db.query("SELECT id FROM reservas WHERE estado = 'em_curso
    '", (err, resultados) => {
            resultados.forEach(r => {
                const ultimo = ultimoHeartbeatSessao.get(r.id);
                if (ultimo === undefined) return; // ainda sem
    sinal -- benefício da dúvida
                if (agora - ultimo <= limiteMs) return;
                terminarSessaoAntecipadamente(r.id, () => {});
            });
        });
    });
}, 30000);
```

Listagem 4.11: Tarefa periódica de deteção de abandono: uma sessão sem heartbeat há mais tempo do que o limite configurado é terminada automaticamente (excerto simplificado).

Uma reserva `em_curso` sem qualquer registo no mapa (sessão acabou de começar e ainda não chegou o primeiro *heartbeat*, ou o processo do *backend* acabou de reiniciar) não é alterada. Nestes casos dá-se o benefício da dúvida até chegar um primeiro sinal, em vez de a terminar por engano. Esta escolha é coerente com a limitação, abordada na Secção 3.1, de o sistema não persistir o estado das suas tarefas

em segundo plano entre reinicializações.

A sequência de término antecipado de sessão — desligar o equipamento, arredondar ao bloco de 10 minutos, marcar a reserva como concluída, revogar o *token* de sessão (Secção 4.8) e notificar a fila de espera da libertação da vaga (Secção 4.10) — está concentrada numa única função, `terminarSessaoAntecipadamente`, invocada a partir de três pontos distintos:

- A rota do aluno na aplicação de agendamento (`/api/cancelar-antecipado`);
- O botão de término de sessão na página do laboratório (`/api/sessao/terminar`, autenticado da mesma forma que o *heartbeat*, já que não tem acesso ao *token* de sessão local do aluno);
- A própria tarefa de deteção de abandono.

Em qualquer um dos três cenários, mantém-se a política de créditos estabelecida para o encerramento antecipado por iniciativa do aluno: o tempo não utilizado não é devolvido.

4.12 Suporte a Múltiplos Postos de Trabalho

Em conformidade com a arquitetura descrita na Secção 3.8, os postos ativos são carregados da base de dados para uma variável em memória (`POSTOS_ATIVOS`) no arranque do processo e recarregados sempre que o administrador cria, edita ou desativa um posto. Esta abordagem evita consultas recorrentes à base de dados em cada pedido que necessite de saber quais os postos disponíveis, exigindo apenas que qualquer alteração aos postos seja efetuada através da própria aplicação.

A escolha automática de posto (Listagem 4.12) reutiliza a mesma instrução SQL de deteção de sobreposição de horários apresentada na Secção 4.3 (Listagem 4.2). A única diferença é pedir `DISTINCT posto_id` em vez de `id`, identificando em memória o primeiro posto ativo cujo identificador não conste no conjunto de postos ocupados durante esse intervalo.

```
1 function escolherPostoLivre(data, horaInicio, horaFim,
      cruzouMeiaNoite, callback) {
2     if (POSTOS_ATIVOS.length === 0) return callback(null, null);
3     db.query(sqlConflitoPorPosto, params, (err, ocupadosRows) => {
4         if (err) return callback(err);
5         const ocupadosIds = new Set(ocupadosRows.map(r => r.
      posto_id));
6         const livre = POSTOS_ATIVOS.find(p => !ocupadosIds.has(p.
      id));
7         callback(null, livre || null); // null: todos os postos
      ativos estão ocupados
```

```
    });
}
```

Listagem 4.12: Escolha automática de um posto livre, reutilizando a instrução SQL de deteção de conflitos já usada na criação de reservas (excerto simplificado).

Esta função é invocada nos três pontos do sistema onde uma reserva é criada: o pedido direto do aluno (`/api/reservar`), a atribuição de uma vaga na fila de espera em tempo real (Secção 4.10) e a criação de reservas extraordinárias pelo administrador. Neste último caso, a integração representou uma melhoria de consistência, visto que a rota de reserva extraordinária utilizava uma verificação simplificada que não tratava adequadamente reservas que cruzavam a meia-noite nem o estado `em_curso`.

A definição de bloco de 10 minutos "ocupado", utilizada transversalmente para construir o calendário e determinar a elegibilidade de entrada na fila de espera, foi generalizada: passou da verificação da existência de pelo menos uma reserva nesse bloco para a existência de reservas em *todos* os postos ativos sobre esse bloco (Listagem 4.13). Caso contrário, um único posto ocupado continuaria a bloquear o horário para os restantes postos.

```
const totalPostos = POSTOS_ATIVOS.length;
const blocosCheios = new Set();
postosPorBloco.forEach((postosOcupados, bloco) => {
    if (totalPostos > 0 && postosOcupados.size >= totalPostos)
    blocosCheios.add(bloco);
});
```

Listagem 4.13: Um bloco só é considerado cheio quando todos os postos ativos têm, nesse bloco, uma reserva (excerto simplificado).

Como consequência, a capacidade total de um dia deixou de ser uma constante fixa (144 blocos de 10 minutos) e passou a ser proporcional ao número de postos ativos ($144 \times N$). Este valor é utilizado tanto no cálculo de estatísticas de ocupação como na coloração do calendário do aluno (Secção 5.2), que passa a consultar o *endpoint* `GET /api/postos-ativos-contagem` — acessível a qualquer utilizador autenticado — para obter o total atual de postos ativos sem conhecer qualquer detalhe sobre cada posto individual.

Por fim, as rotas que anteriormente comunicavam com um único *Raspberry Pi* de endereço fixo — vídeo em direto, estado do *hardware* e as três rotas de controlo direto pelo administrador apresentadas na Secção 4.7 — passaram a resolver o endereço e a porta de destino dinamicamente com base no posto associado ao pedido (`posto_id`). As constantes de endereço fixo do *Raspberry Pi*, usadas na versão anterior do sistema, foram eliminadas: quando uma reserva não tem posto associado — situação possível

apenas em reservas anteriores à introdução do campo `posto_id` (Secção 3.2) — é utilizado o primeiro posto ativo da lista como valor de recurso, conforme ilustrado na Listagem 4.3.

Capítulo 5

Implementação do Frontend

Este capítulo descreve a implementação da aplicação *frontend*, uma SPA *React* responsável pela interface de reserva, pelo painel de controlo de acesso ao laboratório e pelo painel de administração.

5.1 Estrutura da Aplicação

A aplicação foi construída com *React*, na sua versão 19, gerada através da ferramenta *Create React App* (CRA) [19]. Ao contrário da generalidade das aplicações SPA de maior dimensão, este *frontend* não recorre a nenhuma biblioteca de *routing* (como o *React Router*): a navegação entre ecrãs é realizada alternando os componentes que são apresentados no ecrã principal, consoante o estado da autenticação e papel do utilizador autenticado.

O componente de topo, `App.js`, atua como um gestor de rotas simplificado baseado em papéis de acesso: se não existir uma sessão válida, é apresentado o ecrã de *login*; caso o utilizador autenticado detenha privilégios de administrador, é renderizado o painel de administração (`admin_panel.js`). No cenário padrão para alunos, é apresentado o calendário de reservas (`lab_sche.js`) em conjunto com o painel de controlo do laboratório (`ControloLaboratorio.js`).

Os componentes caracterizam-se por serem ficheiros únicos e extensos, sem folhas de estilo próprias ou modularizadas. A formatação visual segue, contudo, duas abordagens distintas: o calendário de reservas (`lab_sche.js`) recorre às classes utilitárias do *Tailwind CSS* [20], carregado diretamente no `index.html`, enquanto o painel de

administração e o painel de controlo aplicam a formatação em linha (*inline styles*). Ambas as abordagens mantêm a lógica e a interface de cada ecrã num único local, mas a coexistência das duas limita a reutilização de estilos entre componentes e constitui uma inconsistência do projeto.

O endereço da API (`API_URL`) e o endereço da página de autenticação do DEE são lidos de variáveis de ambiente (`REACT_APP_API_URL` e `REACT_APP_DEE_LOGIN_URL`), definidas num ficheiro `.env` próprio do *frontend*, mantendo-se o endereço de produção no código apenas como valor de recurso, para que a aplicação continue funcional caso a variável não esteja definida. Note-se, contudo, que o CRA substitui estas variáveis durante a compilação, e não em tempo de execução, pelo que mudar de servidor continua a exigir uma nova compilação (Capítulo 8).

5.2 Calendário de Reservas

O componente `lab_sche.js` suporta a totalidade do fluxo de reserva: seleção de dia num calendário, escolha da hora e da duração a partir dos horários disponíveis nesse dia, e confirmação do pedido junto da API. Após uma reserva bem-sucedida, é apresentado ao aluno um resumo da reserva confirmada (data, horário e duração) e um botão para adicionar o evento ao calendário *Outlook*. Esta funcionalidade utiliza um URL de composição direta do calendário *web* da *Microsoft*, abrindo um novo separador exclusivamente caso o aluno clique na referida opção. A Figura 5.1 apresenta este ecrã.

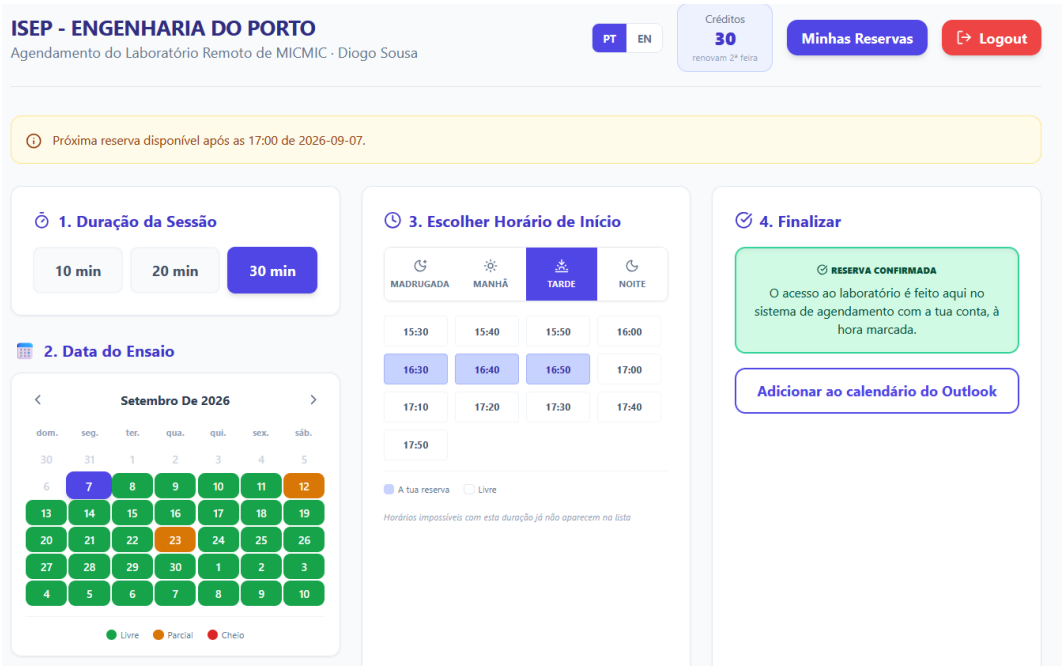


Figura 5.1: Calendário de reservas do aluno: seleção de duração, dia (colorido consoante a disponibilidade) e período do dia, com o resumo de créditos e o seletor de língua no cabeçalho.

Para garantir uma experiência fluida e não intrusiva, a integração com o calendário *Outlook* é estritamente opcional. A confirmação da reserva no sistema ocorre de forma independente, sendo posteriormente disponibilizado um botão para exportar o evento. Este botão encontra-se acessível em dois locais: no quadro de confirmação imediata e junto de cada registo na lista "As Minhas Reservas". A Listagem 5.1 apresenta a função responsável por gerar esta ligação, partilhada por ambas as interfaces.

```
1 const abrirLinkOutlook = ({ date, horaInicio, horaFim, duracaoMin
    }) => {
2   const subject = encodeURIComponent('Reserva Laboratório Remoto')
    ;
3   const body = encodeURIComponent('Duração: ${duracaoMin} min');
4   window.open(
5     'https://outlook.office.com/calendar/0/deeplink/compose?
    subject=${subject}' +
6     '&body=${body}&startdt=${date}T${horaInicio}&enddt=${date}T${
    horaFim}',
7     '_blank'
8   );
9 };
```

Listagem 5.1: Função utilitária, reutilizada em dois pontos da interface, que abre a hiperligação de composição de evento no Outlook Web apenas por ação explícita do utilizador.

Quando a fila de espera por email atribui uma vaga a um aluno (Secção 4.5), o calendário passa a apresentar-lhe o aviso da Figura 5.2, com a contagem decrescente do prazo de marcação.



Figura 5.2: Aviso apresentado ao aluno a quem a fila de espera atribuiu uma vaga (Secção 4.5): o dia fica reservado só para ele durante uma hora, com contagem decrescente do prazo de marcação e o dia assinalado no calendário.

5.2.1 Fila de Espera em Tempo Real

Correspondendo ao mecanismo descrito na Secção 4.10, o próprio componente `lab_sche.js` abre, sempre que o aluno está inscrito na fila de espera do dia atual, uma ligação SSE ao *backend* (`EventSource`), e apresenta, assim que recebe o evento `vaga-disponivel`, uma janela de destaque com o horário e a duração concretos da vaga oferecida e um botão de aceitação imediata — sem que o aluno tenha de voltar a interagir com o calendário. Enquanto aguarda, o aluno vê ainda a sua posição atual na fila, atualizada em tempo real sempre que outro aluno entra, desiste ou avança. Aceitar a oferta invoca `POST /api/fila-tempo-real/aceitar` e, em caso de sucesso, conduz diretamente ao painel de controlo do laboratório (Secção 5.3),

já com uma sessão em curso — sem qualquer ecrã intermédio de confirmação de reserva, ao contrário do fluxo normal de reserva descrito acima.

Enquanto aguarda, o aluno vê o aviso da Figura 5.3, com a sua posição na fila. Quando uma vaga se liberta, esse aviso dá lugar à oferta da Figura 5.4.

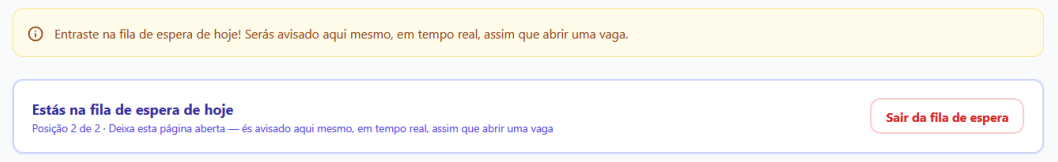


Figura 5.3: Aviso apresentado enquanto o aluno aguarda na fila em tempo real, com a posição atual, atualizada sempre que outro aluno entra, desiste ou avança, e a opção de sair da fila.

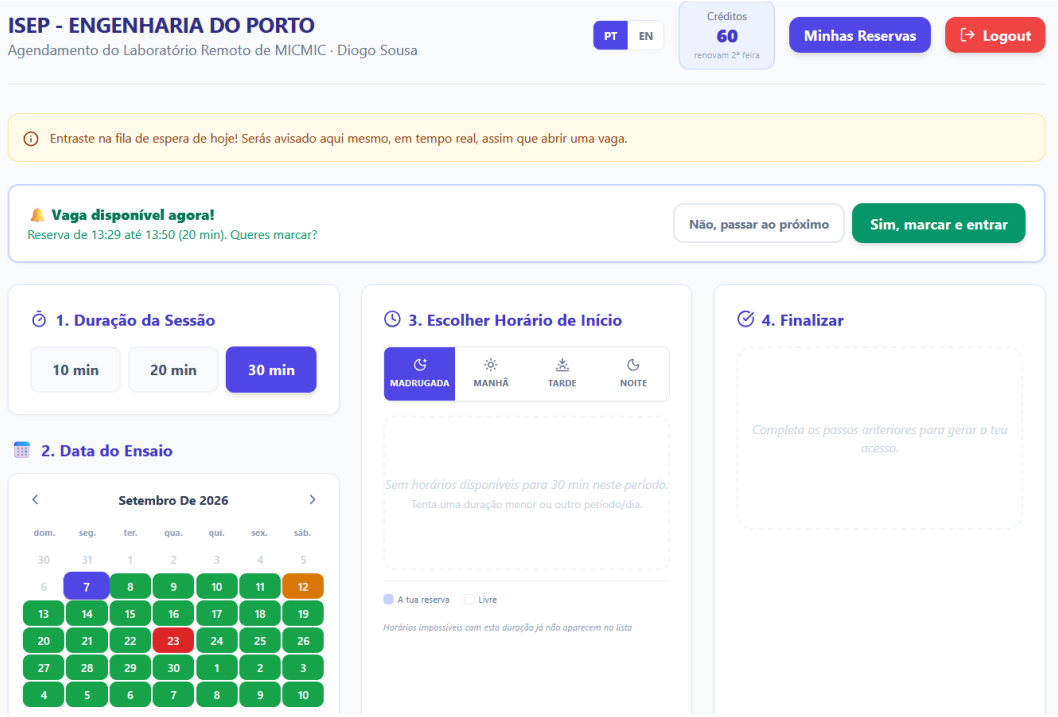


Figura 5.4: Oferta de vaga recebida pelo canal SSE, que substitui o aviso de espera. Indica o horário e a duração concretos da vaga e permite aceitá-la de imediato ou passar ao aluno seguinte.

Enquanto esta ligação estiver aberta, o componente envia também, a cada 30 segundos, um sinal de vida (`POST /api/fila-tempo-real/heartbeat`) para que o *backend* saiba que o aluno continua efetivamente à espera (Secção 4.10) — este sinal é inteiramente automático, sem qualquer ação visível para o aluno, que apenas deixa de ser enviado quando a aba é fechada ou o aluno sai explicitamente da fila.

5.3 Painel de Controlo do Laboratório

O componente `ControloLaboratorio.js` apresenta ao aluno o estado da sua sessão em três fases distintas: sem reserva ativa no momento (`"nenhum"`), em contagem decrescente até ao início de uma reserva já detetada (`"contagem"`), e com a sessão a decorrer (`"ativo"`). Neste último estado, são disponibilizados os comandos de interação, nomeadamente o botão para confirmar a entrada no laboratório (invocando o *endpoint* `/api/confirmar-entrada` descrito na Secção 3.4) e a opção de terminar a sessão antecipadamente. A Figura 5.5 ilustra dois destes estados.



Figura 5.5: Painel de controlo do laboratório nos estados `"contagem"` (em cima, com o botão de entrada bloqueado até à hora reservada) e `"ativo"` (em baixo, sessão já em curso).

A transição entre estes três estados é recalculada a cada segundo pela função `ajustarEstadoTempo` (Listagem 5.2), que replica, do lado do *frontend*, o mesmo cuidado com reservas que atravessam a meia-noite já discutido para o *backend* (Secção 3.4).

```
1  const ajustarEstadoTempo = (horaInicioStr, horaFimStr) => {
2    const agora = new Date();
3    const ano = agora.getFullYear();
4    const mes = String(agora.getMonth() + 1).padStart(2, '0');
5    const dia = String(agora.getDate()).padStart(2, '0');
6    const dataHoje = `${ano}-${mes}-${dia}`;
7
8    const horaInicio = new Date(`${dataHoje}T${horaInicioStr}`);
9    let horaFim = new Date(`${dataHoje}T${horaFimStr}`);
10
11   // Se a reserva cruza a meia-noite (fim <= início), o fim é no
       dia seguinte
12   if (horaFim <= horaInicio) {
13     horaFim.setDate(horaFim.getDate() + 1);
14   }
15
16   if (agora < horaInicio) {
```

```
17      setEstadoAcesso("contagem");
18      setTempoRestante(formatarMilissegundos(horaInicio - agora));
19    } else if (agora >= horaInicio && agora <= horaFim) {
20      setEstadoAcesso("ativo");
21      setTempoRestante("");
22    } else {
23      setEstadoAcesso("nenhum");
24      setReservaAtiva(null);
25    }
26  };
```

Listagem 5.2: Cálculo do estado do painel de controlo (contagem decrescente ou sessão ativa), com tratamento de reservas que atravessam a meia-noite.

Enquanto a sessão se encontra no estado `"ativo"`, um segundo mecanismo verifica periodicamente o acesso junto do *backend*, através de um `setInterval` que invoca repetidamente o *endpoint* `/api/verificar-acesso`. Para otimizar o desempenho e prevenir o envio de pedidos redundantes causados por renderizações no ciclo de vida do *React*, decidiu-se isolar o temporizador através de uma referência (`useRef`) sincronizada com o estado local. Esta alteração permite que o intervalo de consulta leia sempre os dados atualizados da reserva sem forçar a recriação do seu próprio ciclo de execução, estabilizando assim a taxa de pedidos.

O botão de terminar a sessão antecipadamente invoca o *endpoint* `/api/cancelar-antecipado` apresentado na Secção 4.4. Após uma confirmação explícita por parte do utilizador e a respetiva resposta com sucesso do *backend*, a aplicação atualiza o estado local (Listagem 5.3). Esta ação revoga de imediato o acesso visual ao painel de controlo e desencadeia a atualização da lista de reservas do aluno.

```
1  const handleBotaoPanico = async () => {
2    if (!window.confirm(
3      "Tens a certeza que queres terminar a sessão mais cedo? " +
4      "Os blocos seguintes serão libertados, mas os créditos não são
       devolvidos."
5    )) return;
6
7    try {
8      const response = await fetch(`${API_URL}/api/cancelar-
       antecipado`, {
9        method: 'POST',
10       headers: {
11         'Content-Type': 'application/json',
12         'Authorization': `Bearer ${token}`
13       },
```

```
14        body: JSON.stringify({ reserva_id: reservaAtiva.reserva_id
      })
15      });
16
17      const data = await response.json();
18      alert(data.message);
19
20      setReservaAtiva(null);
21      setEstadoAcesso("nenhum");
22      if (carregarReservas) carregarReservas();
23    } catch (error) {
24      alert("Erro ao executar botão de pânico.");
25    }
26 };
```

Listagem 5.3: Ação do botão de pânico: confirmação, pedido ao backend e atualização imediata do estado local.

5.4 Painel de Administração

O componente `admin_panel.js` disponibiliza aos utilizadores com privilégios de administrador a gestão de contas (incluindo a atribuição de créditos e papéis de acesso) e a consulta das reservas do sistema. A interface oferece filtros por estado (nomeadamente o estado de `falta`, detalhado na Secção 4.6) e apresenta um resumo estatístico global contendo o número total de reservas ativas, concluídas, cancelamentos, faltas e utilizadores.

Este painel partilha o mecanismo de autenticação e o endereço da API com o restante *frontend*, diferenciando-se pelas rotas de administração que invoca, as quais se encontram protegidas no *backend* pela função `verificarAdmin` (Secção 3.3). Os parâmetros configuráveis do sistema — o intervalo de escalonamento de notificações T_1 e a janela de verificação da fila de espera (Secção 4.10), o tempo de inatividade tolerado antes de uma sessão ser considerada abandonada (Secção 4.11) e a gestão dos postos de trabalho ativos (Secção 4.12) — foram agrupados num separador designado "Configurações". As Figuras 5.6 a 5.9 apresentam os quatro separadores.

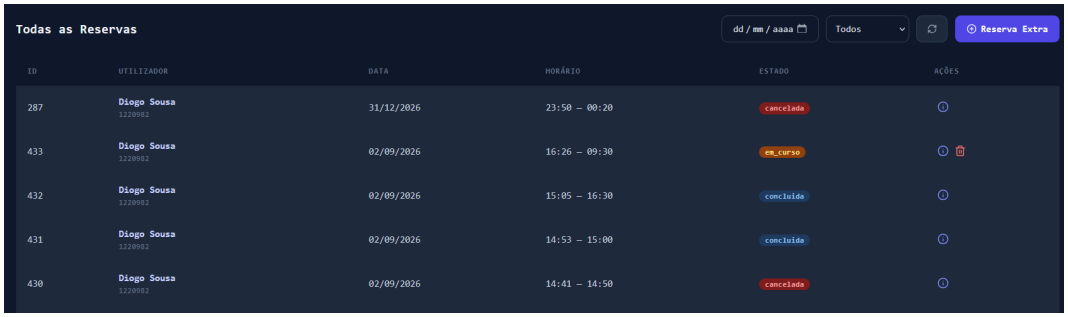


Figura 5.6: Painel de administração — separador "Reservas", com a navegação lateral e a listagem de todas as reservas do sistema, filtrável por data e por estado.

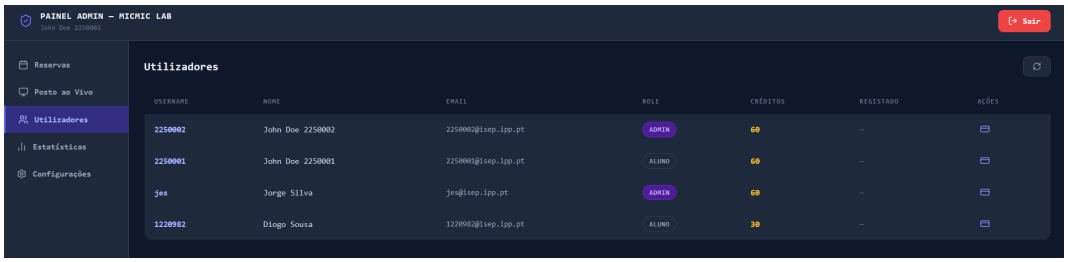


Figura 5.7: Separador "Utilizadores": listagem das contas registadas, com o *username* institucional, o papel atribuído, os créditos disponíveis e a ação de ajuste de créditos.

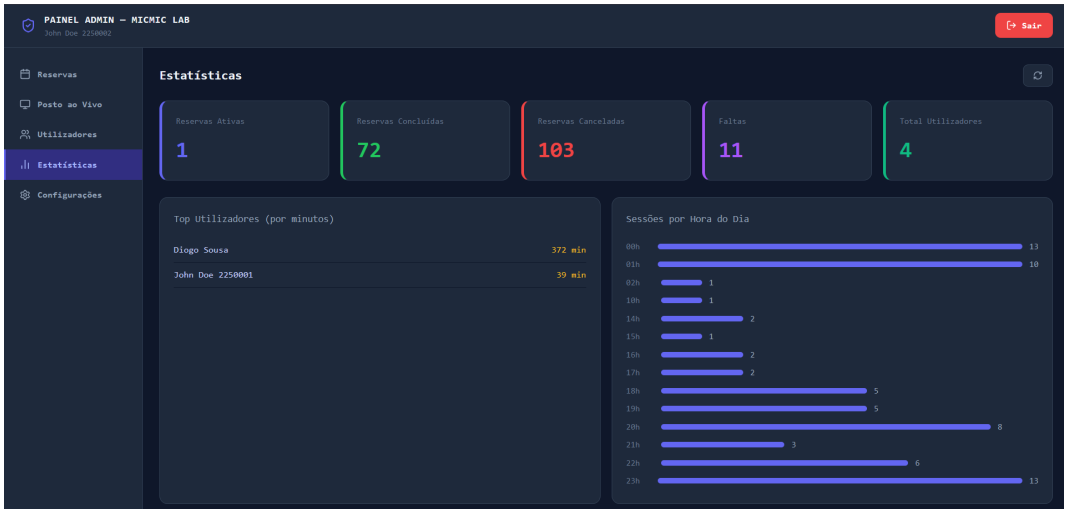


Figura 5.8: Separador "Estatísticas": totais de reservas ativas, concluídas, canceladas, faltas e utilizadores, acompanhados do tempo de utilização por aluno e da distribuição das sessões por hora do dia.

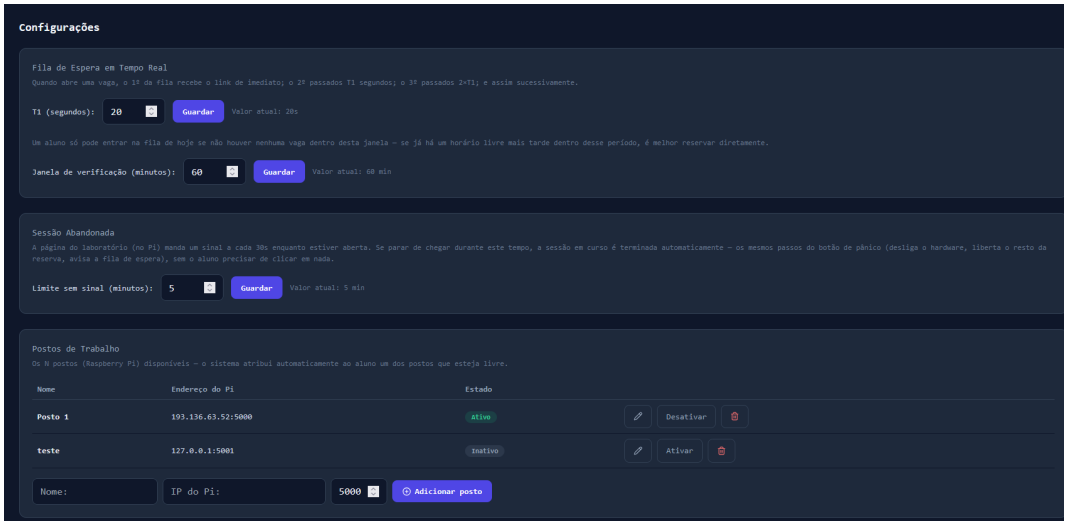


Figura 5.9: Separador "Configurações": parâmetros da fila em tempo real (T_1 e janela de verificação), do limite de sessão abandonada, e gestão dos postos de trabalho.

5.4.1 Acompanhamento da Sessão em Curso e Controlo de Hardware

Com a introdução de múltiplos postos de trabalho (Secção 4.12), o separador "Posto ao Vivo" foi adaptado para gerir várias sessões. A interface passou a integrar um seletor com todos os postos ativos, indicando visualmente quais têm uma sessão em curso e quais estão livres, e mostra automaticamente o conteúdo referente ao posto selecionado (vídeo, estado do *hardware* e comandos), garantindo que o identificador correto (`posto_id`) é incorporado em todos os pedidos remetidos ao *backend* (Figura 5.10).



Figura 5.10: Separador "Posto ao Vivo": vídeo em direto, estado dos interruptores e saída do DAC (em cima), e consola série com histórico e campo de envio (em baixo).

Sempre que existe uma sessão de aluno em curso no posto selecionado, o painel apresenta o vídeo em direto da bancada, o estado lógico dos interruptores, a saída do conversor digital-analógico e a leitura mais recente da porta série. Adicionalmente, permite ao administrador atuar sobre esse mesmo *hardware* (Secção 4.7). Os interruptores funcionam como elementos interativos que alternam o estado da placa ao serem clicados, o valor do conversor digital-analógico passou a ser ajustável através de um controlo deslizante (*slider*) e foi introduzido um formulário para a injeção de comandos de texto diretamente na porta série do equipamento.

A consulta periódica do estado do laboratório está ativa por omissão, mas pode ser desativada pelo administrador. Nesse caso, há um botão que permite obter o estado pontualmente com a atualização automática desligada. Este controlo evita que o painel interrogue o *Raspberry Pi* a cada poucos segundos quando administrador não está a acompanhar a sessão, poupando a carga de processamento analisada em detalhe na Secção 6.5.

5.4.2 Responsividade e Modal de Detalhe de Reserva

O painel de administração foi adaptado para garantir o seu uso em dispositivos com ecrãs de menores dimensões (tipicamente telemóveis). Quando a largura da janela é inferior a um limiar pré-definido, a barra lateral de navegação transita dinamicamente para um formato horizontal com deslocamento (*scroll*).

Adicionalmente, foi implementada uma janela detalhada da reserva, acessível a partir da listagem de reservas. Esta vista inclui os dados gerais da sessão, como a identificação do aluno, o posto de trabalho alocado, o estado atual da reserva e quem foi o responsável pela sua criação (permitindo distinguir marcações de alunos de reservas criadas por administradores). Destaca-se ainda a exposição de métricas temporais detalhadas: a data e hora em que a sessão foi marcada, o horário originalmente reservado, o bloco de tempo efetivamente utilizado (que pode ser inferior em caso de encerramento antecipado) e os instantes exatos de entrada e saída do laboratório.

5.5 Suporte Multilingue

Para acomodar a diversidade linguística dos alunos do ISEP, foi acrescentado suporte para português e inglês em toda a interface principal do aluno (`lab_sche.js` e `ControloLaboratorio.js`). As traduções estão centralizadas num único módulo (`translations.js`), com um dicionário plano por língua, evitando *strings* de texto dispersas pelo código.

Uma vez que os componentes principais da aplicação operam de forma independente e não estão diretamente encadeados, a mudança de idioma não podia ser passada de forma direta. Para contornar isto e para garantir que a alteração do idioma se reflete em toda a aplicação, a preferência do utilizador é guardada no `localStorage` e o sistema emite um evento global (`Event('langchange')`). Assim, sempre que a língua é alterada, os componentes detetam essa mudança de forma independente e atualizam-se de imediato, sem necessidade de dependências complexas.

Capítulo 6

Integração com o Raspberry Pi

O laboratório físico é controlado por uma aplicação *Flask* anteriormente desenvolvida, no âmbito de outro projeto de Projeto/Estágio da LEEC [2]. Esta aplicação está alojada no próprio *Raspberry Pi* e é responsável por comandar as entradas do microcontrolador AVR do sistema usado nas aulas de MICMIC (através de expansores de GPIO e de um conversor digital-analógico ligados por barramento I2C). Para além disso, a aplicação é responsável por disponibilizar vídeo em direto da bancada e por permitir a comunicação série assíncrona com o microcontrolador AVR. Este capítulo descreve a integração entre essa aplicação e o sistema de agendamento, com particular destaque para o mecanismo de acesso direto, autenticado pelo DEE, para a deteção de abandono de sessão e o botão de encerramento antecipado acrescentados à própria página do laboratório.

6.1 Validação de Reservas no Laboratório

As reservas efetuadas no sistema de agendamento constituem a via principal de acesso ao laboratório físico: o aluno, ao clicar em "Entrar no Laboratório" no painel de controlo (Secção 5.3), aciona primeiro a confirmação de entrada junto do *backend*, que devolve um *token* de sessão de curta duração (Secção 3.7). Só depois é que o *browser* é reencaminhado para a aplicação do *Raspberry Pi*, já com esse *token* como parâmetro do URL. A rota principal desta aplicação (`/main`) valida o *token* localmente, confirmando a assinatura com o mesmo segredo partilhado com o *backend*, e o prazo de validade, sem qualquer pedido de rede (Listagem 6.1). O *Raspberry Pi*

não tem, assim, acesso direto à base de dados do sistema de reservas, nem depende de uma chamada síncrona ao *backend* para autorizar cada sessão.

```python
def validar_token_sessao(token):
    if not token:
        return None
    try:
        payload = jwt.decode(token, PI_SESSION_SECRET, algorithms=["HS256"])
    except Exception as e:
        print("Token de sessão inválido:", e)
        return None

    if payload.get('reserva_id') in SESSOES_REVOGADAS:
        return None

    return payload
```

Listagem 6.1: Validação local do token de sessão, incluindo a verificação de revogação (Secção 3.7).

Se o *token* for válido, é guardado na sessão do *browser* (permitindo, por exemplo, recarregar a página sem o reenviar na URL). Em simultâneo, é inicializada uma *thread* em segundo plano dedicada à escuta do canal de revogação da sessão (Listagem 6.2). Caso o *token* seja inválido e não se verifique um acesso direto devidamente autorizado (Secção 6.2), o pedido é imediatamente rejeitado com o código de estado HTTP 403.

```python
def escutar_revogacao(token, reserva_id):
    while reserva_id not in SESSOES_REVOGADAS:
        if not validar_token_sessao(token):
            break  # token expirou entretanto

        try:
            with requests.get(url, params={"token": token}, stream=True, timeout=30) as resposta:
                for linha in resposta.iter_lines(decode_unicode=True):
                    if linha and linha.startswith("data:") and "revoke" in linha:
                        SESSOES_REVOGADAS.add(reserva_id)
                        break
        except Exception as e:
            time.sleep(2)  # ligação caiu -- tenta religar-se
```

Listagem 6.2: Escuta do canal de revogação, com reconexão automática enquanto o token ainda for válido (Secção 4.8, excerto simplificado).

A adoção deste modelo de autenticação baseado em *tokens* responde diretamente à necessidade de garantir a segurança dos recursos do laboratório sem comprometer a usabilidade. A utilização direta de identificadores de utilizador ou chaves estáticas como credenciais representaria uma vulnerabilidade de controlo de acesso, dado que a previsibilidade de um ID sequencial permitiria a uma pessoa maliciosa intercetar ou adivinhar sessões ativas — acedendo, por exemplo, ao *feed* de vídeo ou aos comandos da bancada sem autorização. O sistema atual elimina este risco ao gerar um *token* criptograficamente assinado, imprevisível e com tempo de vida limitado à duração da reserva, assegurando a proteção do sistema.

6.2 Acesso Direto para Utilizadores Autorizados

Para além do fluxo normal de reservas, implementou-se um mecanismo de acesso direto destinado a um conjunto restrito de utilizadores. Esta via permite o acesso imediato ao equipamento, sem necessidade de reserva prévia, mantendo a garantia de que apenas utilizadores autenticados pelo DEE e explicitamente autorizados o podem fazer.

6.2.1 Lista de Utilizadores Autorizados

A lista de utilizadores com privilégios de acesso direto é mantida num ficheiro de configuração próprio da aplicação do *Raspberry Pi* (Listagem 6.3). O isolamento desta configuração em relação à lógica central da aplicação facilita a sua edição futura, dispensando alterações no código-fonte.

```
# pi_config.py
UTILIZADORES_ACESSO_DIRETO = [
    'jes',
    '2250002',
]
```

Listagem 6.3: Ficheiro de configuração com a lista de utilizadores autorizados a aceder diretamente ao laboratório.

6.2.2 Autenticação e Validação do Token do DEE

Quando um utilizador acede à raiz da aplicação (`/`) sem um identificador de utilizador válido, é redirecionado para a página de autenticação do DEE, seguindo o mesmo princípio já descrito na Secção 3.3. Ao regressar, a aplicação recebe um JWT (parâmetro `auth_jwt`), que tem de ser criptograficamente validado antes de confiar na identidade do utilizador (Listagem 6.4).

```
def _obter_chave_publica_dee():
    resposta = requests.get(JWKS_URL, timeout=5)
    jwk = resposta.json()['keys'][0]
    return RSAAlgorithm.from_jwk(json.dumps(jwk))

def validar_jwt_dee(token):
    try:
        chave_publica = _obter_chave_publica_dee()
        return jwt.decode(token, chave_publica, algorithms=["RS512
    "])
    except Exception as e:
        print("Erro ao validar JWT do DEE:", e)
        return None
```

Listagem 6.4: Validação do JWT do DEE na aplicação do Raspberry Pi, obtendo diretamente a chave pública a partir do JWKS.

A lógica de validação teve de ser desenhada à medida para acomodar duas especificidades da implementação JWT do DEE. Em primeiro lugar, o DEE assina os *tokens* com o algoritmo `RS512`, exigindo a explicitação deste parâmetro para prevenir possíveis vulnerabilidades de confusão de algoritmo (Secção 2.3). Em segundo lugar, o cabeçalho dos *tokens* emitidos omite o campo opcional `kid` (*Key ID*). Como as bibliotecas cliente padrão (e.g., `PyJWKClient`) dependem frequentemente deste campo para selecionar a chave correta num JWKS com múltiplas entradas, a sua utilização direta resultaria em falhas de validação. A solução adotada (Listagem 6.4) contorna esta limitação extraindo explicitamente a primeira e única chave disponível no *endpoint* JWKS, convertendo-a diretamente numa chave pública válida.

Após o sucesso desta validação, o identificador extraído do corpo do *token* (*payload*) é cruzado com a lista de utilizadores autorizados (Listagem 6.3). Apenas em caso de correspondência é que a sessão interna do *browser* é marcada com a credencial de acesso direto (Listagem 6.5).

```
def tem_acesso_direto():
    return session.get('utilizador_direto') in
    UTILIZADORES_ACESSO_DIRETO
```

Listagem 6.5: Verificação de acesso direto, baseada exclusivamente na sessão do browser e na lista de utilizadores autorizados.

6.2.3 Independência do Sistema de Reservas

O mecanismo de acesso direto opera de forma independente e paralelo com o sistema de reservas. Esta arquitetura assegura que um utilizador autorizado, caso efetue simultaneamente uma reserva normal, não perde o seu acesso administrativo quando essa reserva termina, nem vice-versa.

6.3 Vídeo em Direto e Leitura da Porta Série

A aplicação do *Raspberry Pi* disponibiliza ainda dois *endpoint*s cruciais para a monitorização da bancada: um fluxo de vídeo da câmara do laboratório no formato *Motion JPEG* (MJPEG) e a leitura contínua da porta série do equipamento. Ambos foram concebidos para integração direta na interface do utilizador, tipicamente através de elementos `<img>` ou requisições *XMLHttpRequest* (XHR).

Numa análise de segurança ao sistema original, constatou-se que estes *endpoint*s se encontravam expostos sem qualquer mecanismo de autenticação. Esta lacuna permitia que pedidos HTTP dirigidos diretamente ao endereço do *Raspberry Pi*, obtivessem acesso ao vídeo em direto e ao registo da porta série, independentemente da existência de uma reserva ativa. Para remover esta vulnerabilidade, aplicaram-se aos fluxos de vídeo e de dados as mesmas regras de proteção e autorização da rota principal (Secção 6.1).

Atualmente, o acesso a estes recursos exige a verificação de um *token* de sessão válido ou a confirmação de privilégios de acesso direto (Secção 6.2), rejeitando qualquer pedido não autorizado com o código de estado HTTP `403` (Listagem 6.6).

```
1 @app.route('/video_feed')
2 def video_feed():
3     token = request.args.get('token')
4     autorizado = bool(validar_token_sessao(token)) or
      tem_acesso_direto()
5     if not autorizado:
6         return jsonify({"erro": "Acesso negado ou sessão inválida/
      expirada."}), 403
7     return Response(gen(pi_camera), mimetype='multipart/x-mixed-
      replace; boundary=frame')
```

Listagem 6.6: Autenticação das rotas de vídeo e de porta série, exigindo um token de sessão válido ou acesso direto.

Esta alteração exigiu também que os modelos HTML que incorporam estes dois *endpoint*s (`main.html` e `terminal.html`) passassem a construir os seus URLs com o *token* incluído (*e.g.*, `url_for('video_feed', token=Token)` em vez de `url_for('video_feed')`). Esta inclusão é necessária porque elementos como a *tag* `<img>` ou pedidos XHR não têm acesso à sessão do *browser* e efetuam requisições independentes que exigem a passagem explícita do *token* como parâmetro de consulta.

6.4 Controlo de Hardware pelo Administrador

Em articulação com o mecanismo implementado no lado do *backend* (Secção 4.7), foram acrescentados à aplicação do *Raspberry Pi* três *endpoint*s (`/api/admin/switch`, `/api/admin/dac` e `/api/admin/serial`) que permitem operar os interruptores, o conversor digital-analógico e a porta série através de pedidos no formato JSON.

Cada uma destes *endpoint*s reutiliza exatamente a mesma lógica de controlo de *hardware* já usada pelos formulários originais da aplicação (respetivamente, `SW_click`, `DAC1` e `TXsend`), acrescentando uma verificação inicial do cabeçalho `X-Admin-Secret` (Listagem 6.7), que tem de corresponder ao mesmo segredo partilhado entre o *backend* e o *Raspberry Pi* (Secção 4.7).

```
1  def pedido_e_do_admin():
2      return request.headers.get('X-Admin-Secret') ==
       PI_ADMIN_SECRET
3
4  @app.route('/api/admin/switch', methods=['POST'])
5  def admin_switch():
6      if not pedido_e_do_admin():
7          return jsonify({"erro": "Não autorizado."}), 403
8      nome = request.json.get('nome')
9      # ... reutiliza a mesma lógica de SW_click ...
10     return jsonify({"sucesso": True})
```

Listagem 6.7: Verificação do segredo partilhado nos endpoints de controlo de hardware do administrador.

Ao contrário dos restantes *endpoint*s de utilização da bancada, estes operam de forma totalmente independente de reservas ativas ou do acesso direto de utilizadores. A autorização baseia-se exclusivamente no segredo administrativo, garantindo que o controlo de gestão sobre o *hardware* permanece isolado do sistema de agendamento de alunos.

6.5 Otimização do Consumo de CPU da Câmara

Durante a validação do sistema em ambiente real, a monitorização efetuada através do comando `top` revelou uma ocupação do processador do *Raspberry Pi* muito superior ao esperado. A investigação deste problema, cujos valores medidos são apresentados no Capítulo 7, identificou três causas distintas na classe `VideoCamera` (`camera.py`), responsável por capturar e codificar o vídeo da câmara:

1. **Captura incondicional:** A captura de vídeo (`PiVideoStream`) era iniciada no arranque da aplicação e mantida permanentemente ativa, independentemente da existência de clientes ligados;

2. **Codificação redundante:** Cada espectador tinha o seu próprio ciclo de codificação *Joint Photographic Experts Group* (JPEG) independente. Com n espectadores em simultâneo, a mesma imagem era codificada n vezes, em vez de uma só;

3. **Ausência de limite de cadência (*rate-limiting*):** O ciclo de codificação operava à velocidade máxima do processador (muito além da capacidade de captura da própria câmara), resultando em grande parte em imagens repetidas, sem qualquer informação visual nova.

Para remover estes problemas, a arquitetura de processamento de vídeo foi reestruturada. Introduziu-se na classe `VideoCamera` um contador de espectadores ativos e a câmara passa a ser ativada apenas no momento da primeira ligação e desativada após a saída do último espectador. Este fecho inclui um período de tolerância programado para evitar reinícios sucessivos em caso de curtas falhas de rede (Listagem 6.8).

A codificação JPEG está, por sua vez, centralizada numa única *thread*, partilhada por todos os clientes (consumidores) através de uma condição de sincronização (`threading.Condition`), o que elimina o processamento redundante associado a múltiplos espectadores. O ciclo de codificação está ainda limitado a um máximo de 15 imagens por segundo (*Frames Per Second* (FPS)), garantindo uma monitorização visual fluida enquanto se reduz drasticamente o esforço computacional.

```
1 def add_viewer(self):
2     with self.lock:
3         self.viewers += 1
4         if self.stop_timer is not None:
5             self.stop_timer.cancel()
6             self.stop_timer = None
7         if self.viewers == 1:
8             self._start_camera()
```

```
9
10 def remove_viewer(self):
11     with self.lock:
12         self.viewers = max(0, self.viewers - 1)
13         if self.viewers == 0:
14             self.stop_timer = threading.Timer(self.
       STOP_GRACE_SECONDS, self._check_stop)
15             self.stop_timer.daemon = True
16             self.stop_timer.start()
```

Listagem 6.8: Gestão do número de espectadores ativos, usada para ligar e desligar a captura de vídeo da câmara (excerto simplificado).

O valor real de imagens por segundo que o ciclo de codificação conseguia atingir foi medido antes de se decidir o limite a impor, através de um contador temporário inserido diretamente no ciclo. Essa medição revelou uma cadência na ordem das 60 imagens por segundo, muito acima do necessário para a monitorização visual da bancada. Esta abordagem de diagnóstico, apesar de simples, permitiu fixar o limite em 15 imagens por segundo com base em dados concretos e não por estimativa, reduzindo para cerca de um quarto o número de codificações JPEG realizadas por segundo.

6.6 Sincronização do Estado do Hardware entre Sessões

A comunicação do estado do *hardware* não se encontrava totalmente sincronizada entre as sessões ativas. As ações executadas pelo aluno na sua interface (`IOinterface.html`) refletiam-se de imediato no painel de administração, graças à consulta periódica já implementada nesse painel (Secção 5.4), mas o inverso não acontecia. Uma alteração efetuada pelo administrador não era refletida na vista do aluno sem um recarregamento manual da página.

Para assegurar a coerência bidirecional do estado da bancada em tempo real, implementou-se na página do aluno um mecanismo de consulta periódica (*polling*) assíncrono com um intervalo de 5 segundos (Listagem 6.9). Esta rotina atualiza automaticamente o estado visual dos interruptores e a posição do controlo deslizante do conversor digital-analógico (*Digital-to-Analog Converter* (DAC)). Para evitar conflitos de utilização durante a interação do aluno, a atualização do DAC é suspensa enquanto o controlo deslizante estiver a ser arrastado.

```
1 setInterval(() => {
2     fetch(`/api/estado-live?utilizador_id=${utilizadorId}&_=${Date
      .now()}`, { cache: "no-store" })
3         .then(r => r.json())
4         .then(estado => {
```

```
5             atualizarIconesInterruptores(estado.switches);
6             if (!arrastandoODac) atualizarPosicaoDac(estado.dac);
7         });
8 }, 5000);
```

Listagem 6.9: Consulta periódica do estado do laboratório na própria página do aluno, com cache-busting para evitar respostas desatualizadas (excerto simplificado).

6.7 Deteção de Abandono a partir da Própria Página do Laboratório

Em articulação com o mecanismo de deteção de abandono implementado no *backend* (Secção 4.11), a página do laboratório (`main.html`) passou a enviar periodicamente um sinal de vida (*heartbeat*). Enquanto a página permanece aberta, é efetuado um pedido HTTP `POST` a cada 30 segundos para o *backend*, utilizando o *token* de sessão já validado (Secção 6.1) para confirmar que a sessão continua em curso (Listagem 6.10).

```
1 var tokenSessao = "{{ Token }}";
2
3 function enviarHeartbeatSessao() {
4     fetch(URL_BACKEND + "/api/sessao/heartbeat", {
5         method: "POST",
6         headers: { "Content-Type": "application/json" },
7         body: JSON.stringify({ token: tokenSessao })
8     });
9 }
10 setInterval(enviarHeartbeatSessao, 30000);
```

Listagem 6.10: Envio periódico de heartbeat a partir da página do laboratório, usando o token de sessão como identificação (excerto simplificado).

Adicionalmente, foi incluído na própria interface um botão para terminar sessão antecipadamente. Este controlo invoca o *endpoint* `/api/sessao/terminar` (Secção 4.11) utilizando o *token* de sessão e permite libertar a bancada sem necessidade de regressar ao painel de agendamento (Secção 5.3). Para garantir visibilidade imediata sem necessidade de navegação vertical (*scrolling*), o botão foi posicionado de forma fixa no canto superior direito do ecrã (`position: fixed; top: 16px; right: 16px`).

Capítulo 7

Testes e Resultados

Este capítulo apresenta os testes realizados ao sistema e os respetivos resultados. São descritos os principais cenários de teste, os problemas identificados durante a validação e os testes realizados no ambiente do DEE.

7.1 Estratégia de Testes

A validação do sistema foi realizada manualmente, através da execução dos principais fluxos da aplicação, incluindo a autenticação, as reservas, o acesso ao laboratório e as operações de administração. Os testes foram realizados com três contas de teste autenticadas através do DEE e abrangeram os diferentes componentes do sistema, incluindo o *backend*, o *frontend* e o *Raspberry Pi*.

7.2 Cenários de Teste de Ponta a Ponta

A Tabela 7.1 resume os principais cenários validados manualmente, cobrindo os três componentes do sistema.

Tabela 7.1: Cenários de teste de ponta a ponta validados manualmente.

Cenário de Teste	Resultado Esperado / Observado
Autenticação e Controlo de Acesso	
Primeiro acesso de um aluno	Redireção para o DEE, criação automática da conta e emissão da sessão local, com o papel correto atribuído.
Acesso direto (autorizado)	Autenticação pelo DEE e acesso imediato ao laboratório sem reserva (Secção 6.2).
Acesso ao *Raspberry Pi* com *token*	*Token* validado localmente pelo *Raspberry Pi*; identificador em bruto é rejeitado (Secção 6.1).
Ações de administração	Gestão de utilizadores, créditos e reservas restrita a contas com papel de administrador.
Gestão de Reservas e Postos	
Reserva de bloco disponível	Reserva confirmada, créditos descontados e resumo da reserva apresentado.
Reserva que atravessa a meia-noite	Bloco identificado no dia seguinte, sem conflito com reservas de outros dias (Secção 4.3).
Acesso com reserva válida	Transição para `em_curso` após confirmação e libertação do acesso no painel.
Terminar sessão antecipadamente	Encerramento da reserva e desligamento do equipamento sem devolução de créditos (Secção 4.4).
Deteção de não comparência	Reserva não utilizada é marcada como `falta`, incluindo a atravessar a meia-noite (Secção 4.6).
Atribuição automática de posto	Associação a um posto livre; bloco indisponível apenas com todos os postos ocupados (Secção 4.12).
Fila de Espera e Concorrência	
Entrada e promoção na fila	Atribuição de vaga pendente ao 1.º aluno com bloqueio do dia (HTTP 423).
Oferta/aceitação em tempo real	Ofertas em cascata com atraso configurado; aceitação cria a reserva num único passo (Secção 4.10).
Sessão em Tempo Real e Hardware	
Controlo de *hardware*	Interruptores, DAC e porta série operados via painel de administração com segredo partilhado (Secção 4.7).
Revogação em tempo real	Botão de pânico revoga o acesso ao vídeo em poucos segundos via canal de eventos (Secção 4.8).
Abandono sem *heartbeat*	Sessão sem sinal de vida da página além do limite é terminada automaticamente (Secção 4.11).

7.3 Problemas Identificados e Corrigidos

A Tabela 7.2 consolida os problemas concretos discutidos ao longo dos capítulos anteriores, com o respetivo estado no final deste projeto.

Tabela 7.2: Principais problemas identificados durante o desenvolvimento e validação do sistema.

Problema	**Componente**	**Estado**
Conflitos de horário entre reservas de dias diferentes (Secção 4.3)	Backend	Corrigido
Equipamento não desligado após o término antecipado de uma sessão (Secção 4.4)	*Backend* / *Raspberry Pi*	Corrigido
Validação insegura do acesso ao *Raspberry Pi* através de um identificador de utilizador (Secção 6.1)	Raspberry Pi	Corrigido
Sobrecarga do processador do Raspberry Pi provocada pela câmara de vídeo (Secção 6.5)	Raspberry Pi	Corrigido
Sessões abandonadas que permaneciam ativas por falta de sinal de vida (Secção 4.11)	Backend	Corrigido
Utilizadores que permaneciam indefinidamente na fila de espera em tempo real após fecharem a página (Secção 4.10)	Backend	Corrigido
Falha na notificação da fila de espera após a libertação de uma vaga (Secção 4.11)	Backend	Corrigido
Verificação incorreta da disponibilidade dentro da janela da fila de espera (Secção 4.10)	Frontend / Backend	Corrigido
Cancelamento antecipado pelo administrador que não libertava corretamente o bloco ocupado (Secção 4.4)	Backend	Corrigido
Verificação simplificada de conflitos na criação de reservas extraordinárias pelo administrador, que não tratava a passagem de meia-noite nem o estado `em_curso` (Secção 4.12)	Backend	Corrigido

7.4 Validação em Ambiente Real

Além dos testes de ponta a ponta descritos na Secção 7.2, foram realizados testes diretamente no ambiente do DEE, incluindo o *Raspberry Pi* utilizado pelo laboratório. Estes testes permitiram validar o funcionamento do sistema e verificar alguns dos comportamentos que dependem da interação entre o *backend*, o *frontend* e o equipamento remoto.

Um dos casos validados em produção foi a otimização do consumo de processador provocado pela câmara do *Raspberry Pi* (Secção 6.5). As medições foram realizadas diretamente no equipamento, através do comando `top`, antes e depois da alteração. Os valores obtidos são apresentados na Tabela 7.3.

Tabela 7.3: Ocupação do processador do Raspberry Pi, medida com `top`, antes e depois da otimização da câmara.

Cenário	**Antes**	**Depois**
Sem qualquer espectador do vídeo	∼10%	∼1%
Um espectador do vídeo	∼100–110%	∼22–34%
Vários espectadores em simultâneo, a interagir com o *hardware*	>300%	∼30–60%

Antes da correção, a câmara mantinha-se ativa mesmo sem utilizadores a visualizar o vídeo, consumindo cerca de 10% do processador. Depois da alteração, esse valor passou para cerca de 1%. Também se verificou uma redução significativa quando existiam vários espectadores em simultâneo, passando de valores superiores a 300% para menos de 60%.

Foi também validado em produção o acesso ao laboratório através de um *token* de sessão (Secção 3.7). Antes da alteração, um pedido direto à rota `/video_feed?utilizador_id=1` permitia obter o vídeo de uma sessão em curso sem uma autenticação adicional. Depois da alteração, esse pedido passou a devolver `403`, sendo necessário um *token* de sessão válido para aceder ao vídeo.

A revogação do acesso foi igualmente testada em produção. Ao terminar uma sessão através do botão de terminar sessão mais cedo, o *Raspberry Pi* recebeu o evento de revogação através do canal de eventos e recusou os pedidos seguintes poucos segundos depois. Desta forma, o acesso não permaneceu válido até ao final do período inicialmente reservado.

O mecanismo de deteção de abandono através de *heartbeat* (Secção 4.11) foi também testado. Numa sessão em que a página do laboratório foi fechada propositadamente, os sinais de vida deixaram de ser recebidos e, após o tempo configurado ter passado, o sistema terminou automaticamente a sessão, desligou o equipamento e marcou a reserva como concluída, sem intervenção manual.

Por fim, a fila de espera em tempo real foi testada simulando dois utilizadores no mesmo dia. Durante este teste foi identificado um problema que fazia com que um utilizador que fechasse a página permanecesse na fila, impedindo o avanço do utilizador seguinte.

Capítulo 8

Conclusões

Este projeto teve como objetivo o desenvolvimento de um sistema completo de agendamento, gestão de reservas e controlo de acesso ao laboratório remoto da Unidade Curricular (UC) de MICMIC da LEEC. A solução desenvolvida integra a infraestrutura de autenticação do DEE, um sistema de créditos e reservas com filas de espera em tempo real e a comunicação segura com a bancada física gerida por um *Raspberry Pi*. Os objetivos definidos no Capítulo 1 foram cumpridos: o sistema encontra-se em funcionamento, foi validado com utilização real e articula com sucesso as três camadas propostas — *frontend*, *backend* e a aplicação do *Raspberry Pi*.

A integração com o mecanismo de SSO do DEE permitiu autenticar os utilizadores com as suas credenciais do departamento, sem necessidade de gestão local de palavras-passe. A validação de acesso no *Raspberry Pi* foi implementada através de JWTs verificados contra o JWKS do DEE, garantindo a autorização das sessões e o controlo do equipamento sem precisar de consultas constantes à base de dados.

A validação em ambiente real permitiu testar a robustez dos algoritmos de agendamento e concorrência, nomeadamente na gestão de reservas, no processamento da fila de espera e no encerramento do equipamento em sessões terminadas mais cedo. Os testes de ponta a ponta confirmaram a eficácia dos mecanismos de *heartbeat* e de revogação de acessos na prevenção do uso indevido da bancada.

8.1 Trabalho Futuro

Como trabalho futuro deste projeto, sugere-se:

- **Penalização por faltas repetidas.** Com o estado `falta`, torna-se possível contar faltas por aluno. Este requisito seria indispensável para um eventual mecanismo de suspensão temporária de alunos com faltas recorrentes;
- **Escolha de posto pelo aluno.** O mecanismo de atribuição automática de posto (Secção 3.8) assume que todos os postos são fisicamente equivalentes entre si, pelo que o aluno nunca escolhe qual pretende usar. Caso o laboratório venha a ter postos com montagens distintas, seria necessário introduzir essa escolha na interface de reserva, em alternativa à atribuição automática atual;
- **Configuração em tempo de execução.** Os parâmetros de implantação são fixados antes da execução, no módulo `config.js` do *backend* (Secção 4.1) e em variáveis de ambiente do *frontend* (Secção 5.1). Mantê-los numa tabela de configurações na base de dados, à semelhança do que já sucede com os postos de trabalho (Secção 3.8), permitiria alterá-los sem recompilar a aplicação nem reiniciar o servidor;
- **Modularização do *backend*.** Dividir o ficheiro único `server.js` (Secção 4.1) em módulos mais pequenos, organizados por domínio funcional (autenticação, reservas, administração), facilitando a manutenção e o desenvolvimento futuro do sistema por mais do que uma pessoa.

Referências

[1] Raspberry Pi Foundation, "Raspberry Pi Documentation," 2024. Acedido em 2026. [Citado na página 1]

[2] A. J. C. d. Sousa, "Laboratório Remoto de MICMIC," relatório de projeto/estágio, licenciatura em engenharia eletrotécnica e de computadores, Instituto Superior de Engenharia do Porto, 6 2025. [Citado nas páginas 2 e 51]

[3] Pallets Projects, "Flask Documentation," 2024. Acedido em 2026. [Citado na página 2]

[4] P. Orduña *et al.*, "The weblab-deusto remote laboratory management system architecture: Achieving scalability, interoperability and federation of remote experimentation," *Cyber-Physical Laboratories in Engineering and Science Education*, 2018. [Citado nas páginas 5 e 6]

[5] V. J. Harward *et al.*, "The ilab shared architecture: A web services infrastructure to build communities of internet accessible laboratories," *Proceedings of the IEEE*, vol. 96, no. 6, 2008. [Citado nas páginas 5 e 6]

[6] SchedMD, "Slurm Workload Manager — Accounting and Resource Limits," 2024. Acedido em 2026. [Citado na página 6]

[7] D. Hardt, "The OAuth 2.0 Authorization Framework." RFC 6749, IETF, 2012. [Citado na página 7]

[8] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)." RFC 7519, IETF, 2015. [Citado na página 7]

[9] OWASP Foundation, "JSON Web Token for Java Cheat Sheet," 2023. Acedido em 2026. [Citado na página 7]

[10] M. Jones, "JSON Web Key (JWK)." RFC 7517, IETF, 2015. [Citado na página 7]

[11] Meta Platforms, Inc., "React - The library for web and native user interfaces," 2024. Acedido em 2026. [Citado na página 7]

[12] OpenJS Foundation, "Node.js Documentation," 2024. Acedido em 2026. [Citado na página 8]

[13] OpenJS Foundation, "Express - Node.js web application framework," 2024. Acedido em 2026. [Citado nas páginas 8 e 20]

[14] Oracle Corporation, "MySQL 8.0 Reference Manual," 2024. Acedido em 2026. [Citado na página 10]

[15] F. S. Riquelme, "jose: JavaScript module for JSON Object Signing and Encryption," 2024. Acedido em 2026. [Citado na página 13]

[16] L. Dusseault, "HTTP Extensions for Web Distributed Authoring and Versioning (WebDAV)." RFC 4918, IETF. `https://www.rfc-editor.org/rfc/rfc4918`, 2007. [Citado na página 14]

[17] WHATWG, "HTML Living Standard — Server-sent events." `https://html.spec.whatwg.org/multipage/server-sent-events.html`, 2026. Acedido em 2026. [Citado na página 14]

[18] Nodemailer, "Nodemailer: envio de email a partir de Node.js." `https://nodemailer.com/`, 2025. Acedido em 2026. [Citado na página 25]

[19] Facebook, Inc., "Create React App," 2023. Acedido em 2026. [Citado na página 39]

[20] Tailwind Labs, "Tailwind CSS: a utility-first CSS framework." `https://tailwindcss.com/`, 2025. Acedido em 2026. [Citado na página 39]

Anexo A

Excertos de Código Complementares

Este anexo reúne dois excertos de código que complementam o Capítulo 6 e a Secção 3.6, apresentados aqui na íntegra para não interromper a narrativa dos capítulos principais.

A.1 Rota de Entrada da Aplicação do Raspberry Pi

A Listagem A.1 apresenta a rota `/` completa da aplicação do *Raspberry Pi*, que decide, em cada pedido, se encaminha o utilizador para o fluxo de reserva (quando existe um *token* de sessão na URL, Secção 3.7), se inicia o fluxo de autenticação do DEE para acesso direto (Secção 6.2), ou se redireciona o utilizador para o *login* institucional, quando nenhum dos dois casos anteriores se aplica.

```
1 @app.route("/")
2 def index():
3     token = request.args.get('token')
4     if token:
5         return redirect(f'/main?page=terminal&token={token}')
6 
7     auth_jwt = request.args.get('auth_jwt') or request.args.get('
    jwt')
8     if auth_jwt:
```

```
        payload = validar_jwt_dee(auth_jwt)
        if not payload:
            return "Acesso negado: token DEE inválido ou expirado.
    ", 403
        username = (payload.get('data') or {}).get('username')
        if username not in UTILIZADORES_ACESSO_DIRETO:
            return ("Acesso negado: este utilizador não está
    autorizado "
                    "para login direto no laboratório."), 403
        session['utilizador_direto'] = username
        return redirect('/main?page=terminal')

    callback_url = request.url_root
    login_url = f"{DEE_LOGIN_URL}?requester={quote(callback_url,
    safe='')}"
    return redirect(login_url)
```

Listagem A.1: Rota de entrada da aplicação Flask do Raspberry Pi.

A Listagem A.2 apresenta o excerto correspondente da rota `/main`, responsável por decidir se um pedido é aceite, com base num *token* de sessão válido (Secção 3.7) ou no acesso direto (Secção 6.2), e por arrancar a escuta de revogação (Secção 6.1) quando o *token* é aceite.

```
token = request.args.get('token') or session.get('token_sessao')
payload = validar_token_sessao(token)

if payload:
    session['token_sessao'] = token
    reserva_id = payload.get('reserva_id')
    if reserva_id not in _threads_escuta_iniciadas:
        _threads_escuta_iniciadas.add(reserva_id)
        threading.Thread(target=escutar_revogacao, args=(token,
    reserva_id), daemon=True).start()
elif tem_acesso_direto():
    session.pop('token_sessao', None)
else:
    session.pop('token_sessao', None)
    return ("<h1>Acesso Negado ou Expirado</h1>"
            "<p>O teu tempo terminou, não tens nenhuma reserva
    ativa, ou não tens "
            "autorização de acesso direto.</p>"), 403
```

Listagem A.2: Verificação de acesso na rota principal da aplicação do laboratório (excerto simplificado).

A.2 Variáveis de Ambiente do Backend

A Listagem A.3 enumera, sem valores reais, as variáveis de ambiente esperadas pelo ficheiro `.env` do *backend* (Secção 3.6), não incluído no controlo de versões do projeto.

```
# Base de dados
DB_HOST=
DB_USER=
DB_PASSWORD=
DB_NAME=

# Autenticação local
JWT_SECRET=

# Rede
APP_URL=
PORT=8080

# Segredo partilhado com o Raspberry Pi (controlo de hardware pelo
    admin)
PI_ADMIN_SECRET=

# Segredo usado para assinar o token de sessão do Raspberry Pi
PI_SESSION_SECRET=

# Envio de email (Nodemailer sobre Gmail)
EMAIL_USER=
EMAIL_PASS=
EMAIL_FROM=
```

Listagem A.3: Estrutura do ficheiro .env do backend (sem valores reais).